



pgeo: a geocoder inside PostgreSQL

Design, tuning, accuracy, capacity and security, measured against Pelias on open data
for the State of Maine

John Zastrow

Version 0.19.0 · 22 September 2026

Contents

Executive summary	1
1. Introduction	3
2. Methods	11
3. Results	24
4. Discussion	64
5. Conclusions	65
6. Future work	67
7. Questions and answers	70
Acknowledgements	72
References	73
Appendix A. Reproducing this study	74
Appendix B. Questions asked during the project	74
Appendix C. pgeo storage	76

Executive summary

This study built and compared two self-hosted geocoders for the State of Maine on the same open data: **Pelias**[1], the established open-source geocoder (Elasticsearch[2] plus five Node.js and C++ services), and **pgeo**, a geocoder written for this project that runs entirely inside PostgreSQL 18 with PostGIS, served either by a thin FastAPI application or, with no application code at all, by PostgREST (“pure SQL”). Pelias served as the reference and as the test oracle for pgeo’s API. Two further engines – **Photon**[3] and **Nominatim**[4] – were built and measured afterwards on the same bench, to test whether Pelias was a hard enough comparison (Sections 3.14 and 3.15).

Accuracy. On 1,560 engine-independent test cases whose answers come from the source data, pgeo answers **95.8%** correctly against **75.5%** for Pelias. The gap is largest where people make mistakes: queries with typos (87% vs 30%), places that do not exist (95% vs 41% handled correctly), lakes and summits (95% vs 41%) and venues (94% vs 68%). Under progressive corruption of 300 base queries, pgeo still answers 75% with two character errors per query, where Pelias answers 10%.

Capacity. Pelias is the faster engine: at every tested CPU size it keeps 4 times as many concurrent users within the latency targets as pgeo (96 against 24 on one vCPU, 384 against 96 on four). pgeo is the smaller one: it meets every target for the 3-user goal of this project in a 1.6 GB memory budget on one vCPU, and holds 24 users there, where the same test needs about 8.3 GB for Pelias in its standard profile and 7.17 GB once every service is squeezed to its floor (Section 3.12). A single missing index had made pgeo's reverse geocoding 20 to 45 times slower; fixing it raised pgeo's one-vCPU capacity from 4 to 24 users. **Optimization.** Those figures describe pgeo as first tuned. Profiling the query path itself afterwards (Section 3.16) cut autocomplete CPU by 36% and raised capacity by a third at every size - 24 to 32 users on one vCPU, 48 to 64 on two, 96 to 128 on four, 128 to 192 unconstrained - on the same memory budgets, narrowing Pelias's lead from four times to three. It was done under a stricter rule than "accuracy held": 7,380 recorded queries return byte-identical results before and after, and a change that would have made autocomplete 3.3 times cheaper was taken back out because it lost one case in 3,360. **The smallest server.** Shrinking one resource at a time until the three-user load stopped meeting its targets (Section 3.12) put pgeo's floor at **0.25 vCPU and 1.40 GB** of memory and Pelias's at **0.25 vCPU and 7.17 GB**. Both idle below a quarter of a core, so at this scale the bill is set by memory: pgeo fits the cheapest shared-CPU plans providers sell, Pelias needs a mid-range server.

Two more engines, and what they change. Pelias is a fair reference but not a sufficient one: pgeo is limited by autocomplete, and the specialist at autocomplete is Photon. Both it and Nominatim were therefore built and measured the same way. On two cores Pelias holds 192 concurrent users, **Photon 128, Nominatim 64 and pgeo 48** - so the cost of putting the whole query path in SQL is four times against Pelias, 2.7 times against Photon, and **1.33 times against Nominatim**, the engine closest to pgeo's own architecture (after the optimization of Section 3.16: three times, two times, and level with Nominatim). Photon did not turn out to be the harsher yardstick: it is quicker per keystroke than pgeo and the lightest engine measured (1.84 GB), but less CPU-efficient than Pelias. On accuracy the two score 73.7% and 61.7% against pgeo's 95.8%, and that gap is mostly data rather than engine - both index OpenStreetMap alone and so cannot hold the E911 address points and Overture places the test set is built from. Two differences are architectural rather than editorial: Nominatim has no autocomplete endpoint (8.0% correct, three-quarters of prefixes returning nothing), and neither engine returns a confidence score, so neither can mark an answer it does not believe.

Compatibility and extensions. pgeo passes all 28 cases of a Pelias API compatibility contract on both front ends, so existing Pelias clients work unchanged, and adds a structured-address endpoint (USPS Publication 28) for the LANCER dispatch application.

Recommendation. For the stated need (a handful of users, a small VM or VPS, messy input, fewest components) pgeo in its pure-SQL form is the better fit; Pelias remains the choice for hundreds of concurrent users per vCPU. Section "Conclusions" maps scenarios to tools.

Key result

On the same open data and the same hardware, pgeo answers 95.8% of 1,560 test queries correctly against 75.5% for Pelias, and it does so in a fifth of the memory at each engine's measured floor. Pelias remains about 4 times faster per CPU. For a small, self-hosted Maine geocoder, accuracy and memory decide, and both favour pgeo.

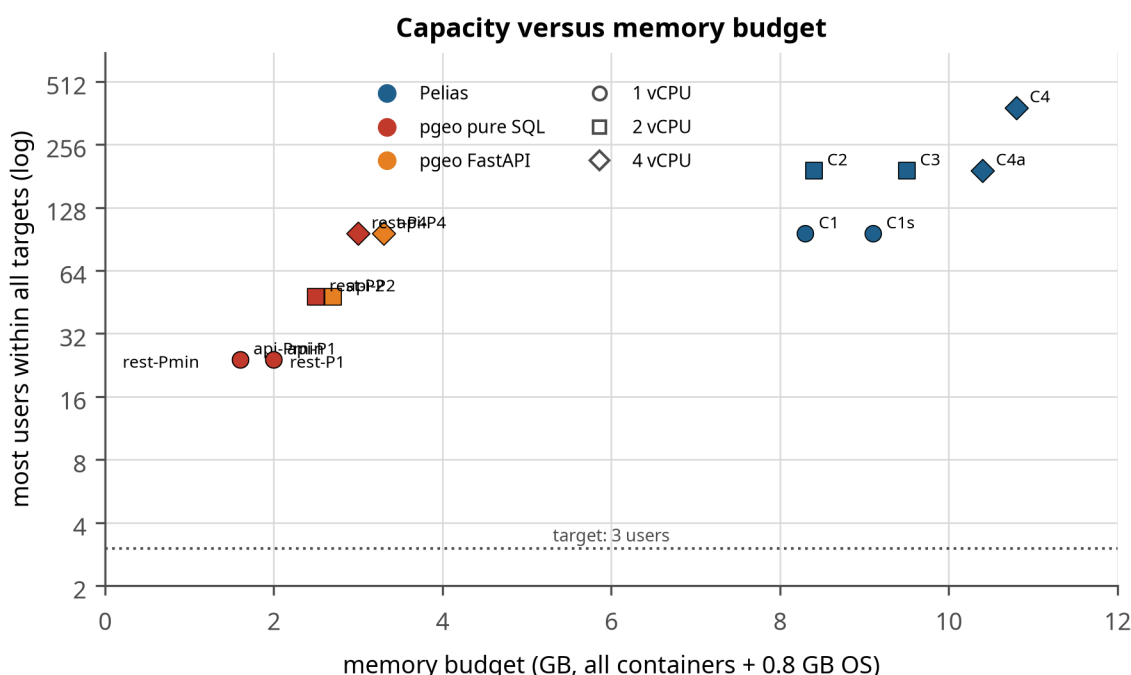


Figure 1. Capacity against memory budget for every tested configuration of both engines. Each point is one configuration; the vertical axis is the most concurrent users for which every endpoint stayed within its latency target (log scale). pgeo configurations sit at the left (1.6-3.3 GB), Pelias at the right (8.3-10.8 GB). The dotted line is the project’s 3-user goal.

Table 1. Key findings.

Question	Answer	Evidence
Which engine is more accurate?	pgeo, by about 20 points overall and 50+ points on typos	Section 3.1
Which is faster under load?	Pelias, four times as many users per CPU	Section 3.4, Table 30
Smallest server for 3 users?	pgeo: 0.25 vCPU, 1.4 GB total. Pelias: see Section 3.12	Section 3.12
Can PostgreSQL alone be the API?	Yes: PostgREST in front of SQL functions, identical accuracy	Sections 2.1, 3.8
Does more data slow Pelias?	Addresses barely; OSM and Overture places cost one ramp step each	Section 3.6
Drop-in replacement for Pelias clients?	Yes for the documented API (27 of 27 contract cases)	Section 3.8

1. Introduction

Why a geocoder was needed, what was already running, what questions the work set out to answer, and what each platform offers. A reader who only wants the outcome can stop after the executive summary; this section explains why the outcome is worth trusting.

1.1 Motivation

Geocoding (turning a typed address or place name into coordinates) and reverse geocoding (turning a map position into an address) are basic services for any application that deals with places. Commercial

geocoding APIs charge per request, send every query to a third party, and change without notice. This project set out to host a feature-rich geocoder for Maine (and later New Hampshire) on the owner's own infrastructure: a Proxmox server on the home LAN, reachable over HTTPS, with a map-based demonstration page.

Two further needs shaped the work. First, the geocoder is meant to feed **LANCER**, a dispatch-and-billing application whose client sites carry sensitive, encrypted street addresses; queries that contain such addresses should not leave the owner's network or linger in logs. Second, a guiding principle (G6) asked for **the stack with the fewest additional components, keeping data in PostgreSQL, unless that produces an inferior or overly complex product**. That principle motivated building pgeo alongside Pelias and measuring both.

The name. *pgeo* is *pg*, the conventional prefix for PostgreSQL's own tools and extensions (*psql*, *pg_dump*, *pg_trgm*), joined to *geo* for geocoding: a geocoder that is PostgreSQL. It is pronounced "pee-geo". The name is this project's own and carries no affiliation with the PostgreSQL project; it is close enough to *pgeocode*, an unrelated Python library for postal-code lookups, to be worth stating that they share nothing but a few letters.

1.2 Starting conditions

Everything below was already in place before the work began. It matters because it bounds the result: the numbers in Section 3 come from this hardware, and a reader on different hardware should expect different absolutes and the same ordering.

Item	State at the start
Infrastructure	Proxmox VE 9.2 node (prox82): AMD Ryzen 5 7600X (Zen 4, 6 cores / 12 threads), 46 GB RAM, guests configured <code>cpu=host</code> with ballooning off; Debian 13 cloud-init template; a TLS-terminating Caddy on the "wharf" VM with public DNS-01 certificates; Forgejo for source control
Target host	VM 120 (geocoder.example.org), 4 vCPU and 10 GB RAM, NVMe storage, LAN only
Build and test host	Workstation: AMD Ryzen 5 3600 6-Core Processor, 12 threads, 31 GB RAM. Two CPU generations behind the Proxmox host (Zen 2 against Zen 4), so a VM core on the host is the faster core: the controlled results in Section 3 are, if anything, conservative for the production VM
Reference recipe	The Pelias docker project for Texas, adapted for Maine
Data	Open data only. No OpenAddresses API token is needed: the statewide Maine file is downloaded without one
Where things run	Builds and all controlled tests run on the workstation; the service runs on VM 120 on the Proxmox host (192.0.2.10) and is reached through the wharf Caddy
Goals	G1 accurate Maine geocoding; G2 LAN-only, HTTPS; G3 reproducible build; G4 demo page; G5 minimum resources for 3 users; G6 fewest components

1.3 Questions

1. How accurate is each engine on Maine data, including misspelled, variant and impossible queries?
2. What are the minimum server resources for 3 concurrent users, and where does each engine break when users are added?
3. How does adding data layers change performance?
4. Can PostgreSQL itself be the API endpoint (G6) without an inferior product?

5. Can pgeo replace Pelias for existing clients, and what does it add?

1.4 What the two platforms do

Before comparing anything it is worth being concrete about what each platform offers, because several of the differences that matter later – structured addresses, typo tolerance, the number of moving parts – are visible here rather than in any measurement.

Table 2. Features of each platform as deployed here.

Capability	Pelias	pgeo
Endpoints	search, structured, autocomplete, reverse, place (+ nearby, beta)	the same five, plus /v1/address
Informational endpoints	/v1/attribution (data licences) and a /v1/ API description page, both HTML	/v1/attribution, built in SQL and served as HTML by both front ends; no /v1/ description page
Filters	layers, sources, focus, rectangle, circle, country, gid, categories	the same
Parsing	libpostal (statistical, ~2 GB model)	rule parser in SQL (best measured); libpostal optional
Typo tolerance	limited (term fuzziness)	trigram similarity on whole names and streets
Town handling	exact admin names (placeholder)	town as a location: villages and nearest towns accepted
Interpolation	separate service (TIGER + OA)	SQL, between neighbouring house numbers
Confidence	per result	per result, lowered for ties between distinct places
Structured US addresses	no	USPS Publication 28 components, delivery and last lines, FIPS codes
Nearest street address for a venue	no	yes (/v1/address)
HTTP layer	Node.js API	PostgREST (no code) or FastAPI
Data store	Elasticsearch index	PostgreSQL tables
Components	6 containers	2-3 containers
Build	importers, ~15 min	one command, ~13 min
Operations tooling	Pelias CLI	tuning profiles, accuracy gate, rebuild script
Security posture here	read-only query services, pinned images, strict CSP; Elasticsearch has no credentials (loopback only)	read-only database role (SELECT only), allowlisted tuning, password-authenticated database, no query parameters in the PostgreSQL log

Two rows carry most of the argument in this report. **Components** is why pgeo fits on a machine Pelias cannot use, and **typo tolerance** is why it answers questions Pelias does not. Everything in Section 3 is an attempt to put numbers on those two lines.

1.5 Pelias, and what this project owes it

pgeo exists because Pelias existed first. Every measurement in this report is made against it, the API pgeo implements is its API, and the test oracle that decided what a correct answer looks like was Pelias itself. That debt deserves stating properly rather than in a footnote.

Pelias[1] began at Mapzen in 2013-2014 as an open-source geocoder built on Elasticsearch, and was released as the hosted Mapzen Search in 2015. When Mapzen shut down in January 2018[5], the hosted services went with it – but the code did not: members of the same team founded Geocode Earth that month and have maintained the project since[6, 7]. It remains actively developed in 2026, under the MIT licence[8], with a community of business and individual contributors.

Three of its design decisions shaped this project directly:

- **Modularity.** Pelias is five services around Elasticsearch – API, placeholder, point-in-polygon, libpostal and interpolation – explicitly so that “you don’t need to be an expert in everything to make changes”. That is the architecture this report measures the cost of, and the cost is real; but the same decomposition is why a reader can understand what each part does, and why Section 3.12 can say exactly which service sets the memory floor.
- **A documented, stable HTTP API.** Pelias published an API worth copying: coherent, GeoJSON, with a geocoding block that carries query, engine and errors. pgeo did not design an interface; it implemented this one, which is why an existing client can be repointed without changes.
- **Open data as a first-class input.** Pelias’s importers for OpenAddresses, OpenStreetMap, Who’s On First and CSV are what make a multi-source geocoder tractable at all. pgeo reads the same sources, and in two cases reads them *through* Pelias’s own tooling: the interpolation database and the OpenAddresses conversion.

Where this report finds pgeo ahead – accuracy on messy input, memory footprint, component count – it is ahead on a problem Pelias defined, measured with a test set whose correct answers were agreed by running Pelias. Where Pelias is ahead, on throughput per CPU and on coverage beyond one state, it is ahead by design decisions that this project chose not to make rather than failed to match. A fair summary is that pgeo is a specialisation: it trades the generality Pelias was built for against a much smaller machine, for one state, for a handful of users.

1.5.1 Is Pelias the right thing to measure against? Worth asking directly, because the whole study rests on the answer.

For features and functions, yes, and by some distance. Pelias is the only open-source geocoder that shares pgeo’s central premise – several independent open sources merged into one index. Nominatim[4] and Photon[3] are OpenStreetMap only; the TIGER geocoder[9] is US Census only; pgGeocoder[10] is Japanese government data only. Pelias imports OpenStreetMap, OpenAddresses, Who’s On First, Geonames, polylines and arbitrary CSV, which is exactly the problem pgeo set out to solve. It also has the widest API surface of the group: search, structured search, autocomplete, reverse and place.

For flexibility, yes. Pelias is the most configurable of the alternatives – custom importers, private datasets, and an explicit goal of working “equally well for a small city and the entire planet”. Measuring against the most adaptable option is the harder test, which is the right one to set.

For scalability, only partly, and the gap is specific. Pelias is a fair reference for concurrent users on one machine, which is what this project needed. It is not the reference for either extreme. At the top end, Nominatim’s planet deployment (roughly a terabyte of disk and 128 GB-class memory) and Photon’s prebuilt planet index define what large means. More importantly for this study, **Photon exists because Nominatim is weak at type-ahead, and it is optimised for autocomplete in tens of milliseconds** – and autocomplete is precisely the endpoint that limits pgeo on every configuration tested (Figure 14). That made Photon the comparison most likely to embarrass pgeo, so it was built and measured rather than left as a caveat; Section 3.14 reports the result. In summary: Photon is about twice as fast per autocomplete request as pgeo and holds 2.7 times as many concurrent users on the same two cores, but it is *less* CPU-efficient than Pelias, not more, so the four-times figure below remains the binding one. The reason is that Photon’s index is OpenSearch, like Pelias’s Elasticsearch, rather than anything special about type-ahead.

Key result

Photon has now been measured (Section 3.14), and it changes the caveat rather than removing it. On equal CPU Photon holds fewer concurrent users than Pelias, so “four times fewer users per CPU than Pelias” is still the widest honest gap for pgeo. What Photon does show is a real per-request advantage at type-ahead, on a fraction of the memory – and a Maine index that is missing most of the data this project cares about, because it can only be built from OpenStreetMap.

Photon also changes the build comparison rather than just the query one: it distributes a prebuilt index, so an operator downloads rather than builds, and the 5.5 GB build machine this report says pgeo needs (Section 3.5) has no Photon equivalent. That is a different trade from the one measured here, and a real one. It comes with a constraint that Section 3.14 quantifies: a Photon index is built from a Nominatim database, and Nominatim imports OpenStreetMap, so the OpenAddresses E911 points and the Overture places that both other engines carry cannot be in it.

1.6 Other geocoders that live in PostgreSQL

pgeo is not the first attempt to put geocoding inside the database, and the question of whether it was worth building depends on what already exists. A survey of the field in September 2026 found three projects that keep their query logic in PostgreSQL, and one that keeps half of it there.

Project	Where the logic lives	Data	Interface	Notes
---------	-----------------------	------	-----------	-------

Table 3. Geocoders whose query logic runs inside PostgreSQL, surveyed September 2026.

Project	Where the logic lives	Data	Interface	Notes
PostGIS TIGER Geocoder[9] (<code>postgis_tiger_geocoder</code>)	entirely PL/pgSQL, plus <code>address_standardizer</code> (PAGC) for parsing	US Census TIGER only	SQL functions; no HTTP	The canonical answer. Released independently of PostGIS since 2025.1 (June 2026)[11], needs PostgreSQL 16+. Exact-match oriented; its own documentation notes speed problems in the <code>address_standardizer</code> [12] wrapper for batch work
osmgeocoder[13]	“primarily as SQL functions”, with Python for orchestration	OpenStreetMap (<code>imposm3</code>), optionally OpenAddresses	Python library plus an optional Flask service	The closest relative: same extensions as pgeo (PostGIS, <code>pg_trgm</code> , <code>fuzzystrmatch</code> [14]). Multi-country address formatting via OpenCage templates. Without <code>libpostal</code> it degrades to “street names only”
pgGeocoder[10]	entirely PostgreSQL functions	Japanese government address datasets (ISJ, KSJ)	SQL only: <code>geocoder('...')</code> and <code>reverse_geocoder(lon, lat)</code>	The closest architectural sibling found: all logic in the database, forward and reverse, PostGIS only. Japan-only, and no HTTP layer
TIGER packagings[15, 16]	none of their own	TIGER	psql; one adds a Python API and Docker	Two of them, both wrapping the PostGIS TIGER geocoder rather than adding matching; both quiet since 2022
Nominatim[4]	indexing and address computation in PL/pgSQL triggers; search in a Python application	OpenStreetMap	HTTP API	Postgres-centric rather than Postgres-only. Version 5 completed the move from PHP to Python. Planet-scale: ~1 TB of disk and 128 GB-class memory; the 2025 roadmap[17] notes search indexes have grown large enough to slow lookups

Table 4. The same feature comparison as Table 2, extended to the other geocoders that keep their logic in PostgreSQL. Compiled from project documentation, September 2026; not independently benchmarked.

Feature	TIGER geocoder	pgGeocoder	osmgeocoder	Nominatim	pgeo
Forward search	yes	yes	yes	yes	yes
Reverse geocoding	yes	yes	yes	yes	yes
Autocomplete / type-ahead	no	no	yes (predictive text)	no (not its strength)	yes
Structured search fields	yes	yes	yes	yes	yes
Typo tolerance	no (exact match)	no (exact match)	yes (trigram, metaphone)	limited	yes (trigram + abbreviation expansion)
Data sources	US Census TIGER only	Japanese government datasets	OSM, optionally OpenAddresses	OSM	OA, OSM, WOF, GNIS, ZCTA, Overture
HTTP API	none (SQL only)	none (SQL only)	Flask service	yes	yes
Runs with no application code	n/a (no HTTP)	n/a (no HTTP)	no	no	yes (PostgREST)
Speaks another geocoder's API	no	no	no	its own	yes (Pelias, 28/28)
Structured postal addresses out	partial (normalizer)	Japanese address parts	formatting templates	no	yes (USPS Publication 28)
Coverage	United States	Japan	many countries	planet	one state, tested
External model needed	no	no	libpostal recommended	no	no (rule parser)

Where pgeo sits. Against the TIGER geocoder, pgeo adds sources beyond TIGER (OpenAddresses, OpenStreetMap, Who's On First, GNIS, ZCTA, Overture), typo tolerance rather than exact matching, and an HTTP API. Against osmgeocoder – which reaches for the same three extensions, and is the nearest thing to a sibling – pgeo needs no application code at all, answers a documented third-party API contract, and returns USPS Publication 28 addresses; osmgeocoder in exchange covers many countries where pgeo covers one state, and is the older and more general project. Against Nominatim, the difference is architectural: Nominatim computes addresses in the database but searches from a Python application, where pgeo's search *is* the database. That difference is now measured rather than asserted – Section 3.15 puts both on the same two cores, and pgeo holds 48 concurrent users to Nominatim's 64. Moving the last of the search into SQL costs a third of the capacity, which is the narrowest gap in this report.

pgeo is therefore not the first geocoder whose query path is entirely SQL: the TIGER geocoder and pgGeocoder both are, and pgGeocoder in particular does forward and reverse geocoding from PostgreSQL functions with nothing else running. What none of them has is an **HTTP interface with no application code** – both are SQL-only, so a client needs a database connection rather than a URL – or an API another geocoder's clients already speak. That is the narrower claim this project can make, and it is narrow: the pieces are all well known – trigram matching, PL/pgSQL, PostgREST – and assembling them this way is uncommon rather than novel.

One nearby category is worth distinguishing, because it is easy to confuse. A foreign data wrapper such as OpenCage's[18] also lets you geocode “from SQL”, but by calling a remote service: the query leaves the building and the privacy argument in Section 1.1 disappears. Nothing in this survey treats that as self-hosted geocoding.

What this means

The interesting comparison is not pgeo against Pelias but pgeo against the alternative inside PostgreSQL. The TIGER geocoder is exact-match and US-Census-only; osmgeocoder needs libpostal and a Python service to do more than street names. A reader who wants typo tolerance, several open data sources and a Pelias-shaped API without an application layer has, as far as this survey found, no off-the-shelf option – which is the gap this project fills, for one state.

1.7 Terms and acronyms

Geocoding borrows vocabulary from several fields, and this report adds names of its own for the configurations it tested. Everything used later is defined here so no term arrives unexplained.

Table 5. Terms and acronyms used in this report.

Term	Meaning
Geocoding	Turning a typed address or place name into coordinates
Reverse geocoding	Turning coordinates into the nearest address or place
Autocomplete	Type-ahead search: results after every keystroke, against a much tighter latency budget
Structured search	A search whose parts arrive in separate fields (address, locality, region, postal code) rather than one string
pgeo	The geocoder built for this project: PostgreSQL and PostGIS, with the query logic in SQL. From pg (PostgreSQL's conventional prefix) and <i>geo</i> ; said “pee-geo”
Pelias	The established open-source geocoder used here as the reference and the test oracle: an API service, Elasticsearch and four helper services
PostGIS	The PostgreSQL extension that adds geographic types, indexes and functions
PostgREST	A gateway that exposes PostgreSQL functions as a REST API, with no application code of its own
FastAPI	A Python web framework; pgeo's alternative front end, kept as a baseline
libpostal [19]	A statistical address parser (a ~2 GB model) used by Pelias and optionally by pgeo
Elasticsearch	The search engine Pelias stores its index in
k6 [20]	The load-testing tool used throughout: it runs scripted virtual users against an HTTP API and reports latency percentiles and error rates (tests/load/k6)
Virtual user (VU)	One simulated person in a load test, issuing requests with pauses between them, as a person would
p95	The 95th-percentile response time: 95 of 100 requests were at least this fast. Used instead of an average because the slow tail is what a user notices
SLO	Service level objective: the latency and error targets a configuration must meet to count as passing (Table 19)
Ramp	Increasing the number of virtual users step by step until a target is missed, to find a configuration's capacity
Breaking point	The step at which a configuration exceeds five times a target, produces more than 5% errors, or a service dies

Term	Meaning
Trigram	A three-character slice of a word[21]. Comparing the sets of trigrams in two strings measures similarity, which is how pgeo tolerates typos (<code>pg_trgm</code>)
WOF	Who's On First[22], the open gazetteer of administrative places used for towns, counties and neighbourhoods
OA	OpenAddresses[23], the open collection of address points
GNIS	The USGS Geographic Names Information System[24]: lakes, summits, streams and populated places
ZCTA	ZIP Code Tabulation Area[25], the US Census approximation of a ZIP code as an area
gid	A globally unique identifier for a record in a geocoder's response, for example <code>whosonfirst:locality:85948877</code>
C1, C2, C4, M0	Names for the Pelias configurations tested: the digit is the vCPU count, M0 is unconstrained (Table 20)
Pmin, P1, P2, P4, PM	The equivalent pgeo configurations: Pmin is the smallest budget tested, PM unconstrained
E1, E2, E4	pgeo configurations given the <i>same memory budget</i> as the Pelias configuration at that CPU size, for an equal-resources comparison (Section 3.4)

2. Methods

How both systems were built and how they were measured. The detail here is what makes Section 3 checkable: the same data, the same load generator, the same targets and the same host for both engines, with every departure from that noted.

2.1 Systems built

The two architectures, and the reason the comparison is not simply one product against another: one is an established multi-service geocoder, the other was written for this project to test whether the database alone is enough.

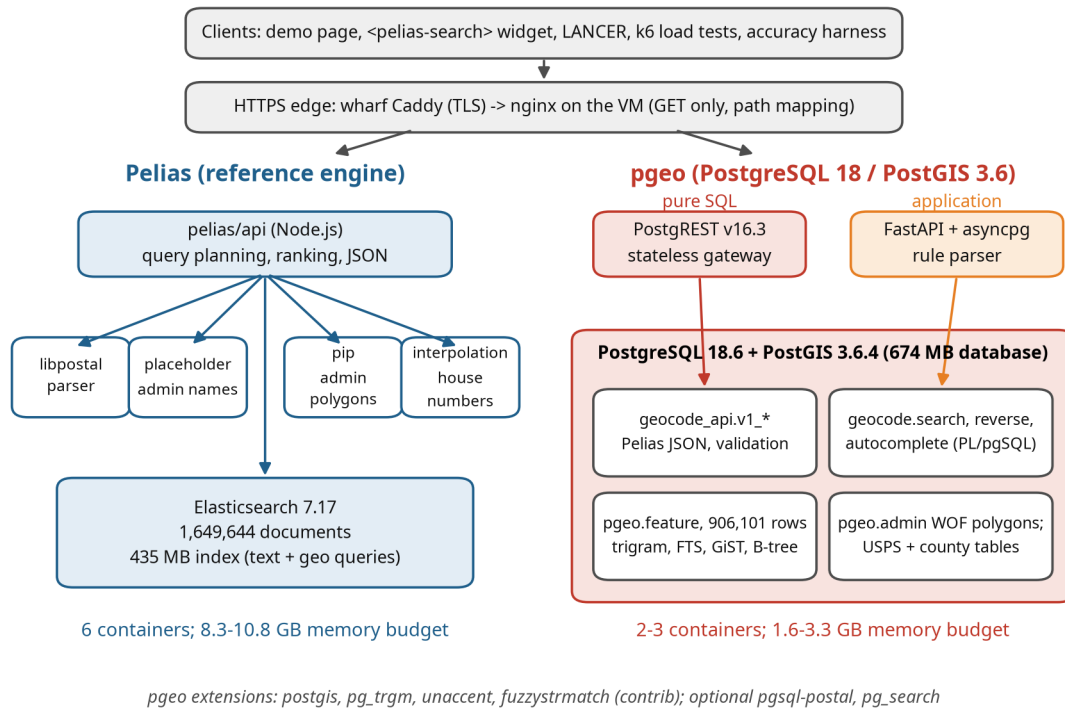


Figure 2. System architecture. Clients reach both engines through the same HTTPS edge. Pelias (left) is an API service coordinating four helper services and Elasticsearch. pgeo (right) keeps data, parsing, ranking and JSON formatting inside PostgreSQL; the HTTP front end is either PostgREST (a stateless gateway, no application code) or a thin FastAPI application.

Figure 2 contrasts the two designs, and the contrast explains most of the results that follow. Pelias divides the work among six processes in three languages, so each part can be scaled or replaced, and Elasticsearch does the text matching in compiled code. pgeo puts everything in one process family: the query, the ranking and the JSON all happen inside the database. That is why pgeo needs a quarter of the memory and fewer moving parts, and why it costs more CPU per request.

Pelias was deployed from pinned images (API, libpostal parser, placeholder, point-in-polygon, interpolation, Elasticsearch 7.17.27) with the index built on the workstation and shipped to VM 120 as an Elasticsearch snapshot. Nginx on the VM serves the demo page and the API under strict security headers; the wharf Caddy terminates TLS.

pgeo runs on PostgreSQL 18.6 with PostGIS 3.6.4 and only core and contrib extensions (`pg_trgm` for trigram similarity, `unaccent`, `fuzzystrmatch`). Every query step is a SQL or PL/pgSQL function (Figure 3 shows the forward-search path). The public API is a schema of functions (`geocode_api.v1_search`, `v1_reverse`, ...) that return Pelias-shaped GeoJSON, so any HTTP layer that can call a function can serve it. Two front ends were built and measured:

- **PostgREST** (pure SQL): maps GET `/rpc/v1_search?text=...` to the function; an nginx edge maps Pelias paths (`/v1/search`) to RPC paths. Chosen over an in-database HTTP server (Omnigres) because it keeps third-party code out of the database process, and over OpenResty because it needs no custom code (docs/HTTP_OPTIONS.md).
- **FastAPI** (application): the same SQL functions behind a Python service, kept as the baseline to show what an application layer adds or costs.

PostgreSQL core and its contrib extensions contain no HTTP server, so “Postgres as the endpoint” always needs either third-party code inside the database or a small gateway beside it. The options were weighed

as follows (docs/HTTP_OPTIONS.md has the full research and sources):

Table 6. HTTP options for serving the SQL API.

Option	For	Against	Decision
PostgREST (external gateway)	no business logic outside SQL; mature; database roles as the security model; crash-isolated from the database; pooling built in	one more process; /rpc/ paths and dotted parameters need an edge mapping	chosen
Omnigres omni_httpd (in the database)	literally "Postgres is the API"; no extra hop; full URL control	third-party C code in the database process; early maturity; upgrades tied to PostgreSQL releases; HTTP workers compete with queries	rejected
OpenResty + pgmoon (nginx + Lua)	nginx already present; full control of paths and parameters	a custom nginx build and Lua code to own	fallback
pg_featureserv, pREST, GraphQL gateways	publish functions quickly	OGC or GraphQL shapes, not the Pelias API	not suitable
FastAPI (application)	flexible; any parser	application code to maintain	kept as baseline

Every step runs inside PostgreSQL; the FastAPI and PostgREST front ends pass parameters in and JSON out

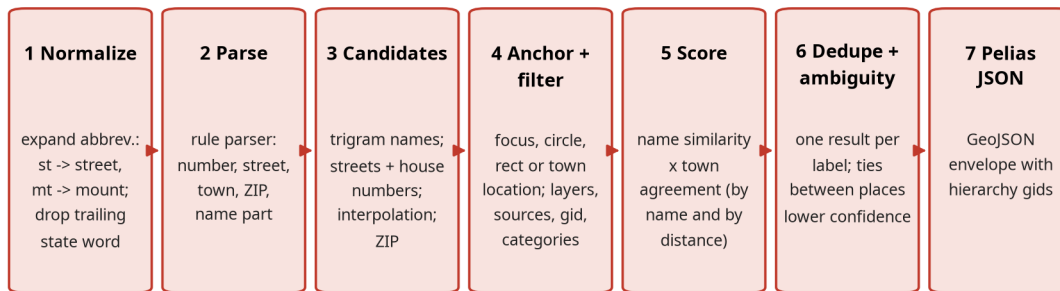


Figure 3. The pgeo forward-search pipeline. Each step is SQL; the tuning work in Section 2.7 changed steps 1, 3, 4, 5 and 6.

Figure 3 matters for anyone who wants to change the ranking: each step is plain SQL in a file under version control, so a change is a pull request and the accuracy gate measures it. In Pelias the equivalent logic is spread across the API service's query builders and Elasticsearch scoring.

2.1.1 What pgeo is made of The case for pgeo is that it is small, so the claim deserves an itemised bill of materials rather than a summary. This section lists every component pgeo is built from, separated by when it is needed: at run time (installed on the server that answers queries), at build time (needed once, to turn raw data into an index, and never again until the next rebuild), and everything else that is convenience rather than requirement. The split matters for a small deployment, because only the first list has to fit on the VPS.

Run time. Answering a query needs a database, three contrib extensions, PostGIS and one HTTP front end. Nothing else runs.

Table 7. pgeo's run-time components: what must be installed on a server that answers queries. The FastAPI front end (below) is an alternative to PostgREST, not an addition.

Component	Version	Licence	Role	Where it runs
PostgreSQL	18.6	PostgreSQL Licence	Storage, query planning, and - through PL/pgSQL - the parsing, ranking and JSON formatting that other geocoders put in application code	the database process
PostGIS	3.6.4	GPL-2.0-or-later	Geometry type, spatial index (GiST), point-in-polygon admin lookup, distance ranking, reverse geocoding	extension, in the database
pg_trgm	ships with PostgreSQL	PostgreSQL Licence	Trigram similarity and the GIN indexes behind fuzzy and type-ahead matching	contrib extension
fuzzystrmatch	ships with PostgreSQL	PostgreSQL Licence	Levenshtein distance and double metaphone, used to rescue typos	contrib extension
unaccent	ships with PostgreSQL	PostgreSQL Licence	Diacritic folding during normalisation	contrib extension
PostgREST	v16.3	MIT	The pure-SQL front end: turns GET /rpc/v1_search into a function call, with database roles as the authorisation model. No application code	a separate process beside the database
nginx	distribution package	BSD-2-Clause	Maps Pelias paths to RPC paths, applies security headers and rate limits, serves the demo page	a separate process

Three of the seven entries ship inside PostgreSQL itself, so a stock `postgis/postgis` image plus PostgREST is the whole run-time stack; in the deployed configuration that is two containers, both pinned by digest (`postgis/postgis:18-3.6` and `postgrest/postgrest:v16.3`). The one dependency worth naming separately is PostGIS, which is GPL-2.0-or-later and by far the largest single component: it is a library loaded into the database process rather than distributed software, so it does not affect the licence of pgeo's own code, but an organisation with a policy on copyleft should know it is there.

The optional application front end. FastAPI was kept as a measured baseline - what an application layer adds and costs - and it is the front end to choose if you need a parser or a response shape that SQL cannot express conveniently. Choosing it replaces PostgREST and adds a Python runtime:

Table 8. The FastAPI front end, an alternative to PostgREST. Pinned exactly in `pgeo/pyproject.toml`.

Component	Version	Licence	Role
Python	3.12 or newer	PSF	Runtime for the application front end
FastAPI	0.141.1	MIT	HTTP routing, request validation, OpenAPI
Uvicorn	0.53.0	BSD-3-Clause	ASGI server
asyncpg	0.31.0	Apache-2.0	PostgreSQL driver and connection pool
Pydantic	2.13.5	MIT	Request and response models
httpx	0.28.1	BSD-3-Clause	Outbound HTTP, used by the loader and tests

Build time. Building the index needs two more tools, neither of which is installed on the serving machine in the recommended deployment - the build runs elsewhere and ships a dump or a data directory:

Table 9. Build-time tools. Needed to make an index, not to serve one.

Component	Version	Licence	Role
DuckDB	1.5.5	MIT	Reads Overture Parquet directly from object storage and the CSV sources, and writes the staging CSVs
GDAL / ogr2ogr	distribution package	MIT	Loads the OpenStreetMap PBF extract and the Census shapefiles into staging tables

Code written for this project. Everything above is off the shelf. What is new is 2,033 lines of SQL and PL/pgSQL in eight files, plus 1,461 lines of Python for loading and the optional front end. The SQL is the geocoder; the Python moves data into it and, in the PostgREST configuration, is absent at run time entirely.

Table 10. The code written for this project. The eight SQL files are the geocoder; the Python is loading, tuning and the optional front end.

File	Lines	What it is
sql/010_base.sql	81	Extensions, schemas, roles, shared helpers
sql/020_tables.sql	115	The tables for one build, created in <code>pgeo_build</code>
sql/022_stage.sql	32	Staging tables to admin boundaries and raw features
sql/025_osm.sql	81	OpenStreetMap rules: named POIs to venues, <code>addr:*</code> to addresses, named roads to streets
sql/030_enrich_index.sql	160	Admin hierarchy by point-in-polygon, normalisation, labels, importance, dedupe, indexes
sql/040_functions.sql	623	The query pipeline: candidate generation, scoring, ranking

File	Lines	What it is
sql/050_api.sql	481	The Pelias-shaped API: geocode_api.v1_search, v1_autocomplete, v1_reverse, v1_attribution, and the PL/pgSQL rule parser
sql/060_address.sql	460	Structured US addresses in the style of USPS Publication 28
src/pgeo/load/	390	The loader: source extraction, build orchestration, atomic schema swap
src/pgeo/api/	603	The optional FastAPI front end and its rule parser
src/pgeo/tune.py	409	The tuning harness used in Section 2.7
src/pgeo/settings.py	59	Configuration and DSN assembly

What is not in the list. pgeo needs no search engine, no JVM, no Node runtime, no message queue, no cache tier and no separate parsing, point-in-polygon or interpolation services. The functions those services provide in Pelias are covered by PostGIS and by the SQL in Table 10 - the admin hierarchy that Pelias's pip service supplies is a spatial join in 030_enrich_index.sql, and the address parsing that libpostal supplies is a rule parser in 050_api.sql. That substitution is not free: it is a rule-based parser standing in for a statistical one, which Section 3.1 quantifies. It is, however, the reason the run-time list is seven entries long and three of them ship with PostgreSQL.

Demonstration and testing. Neither list below is required to run a geocoder; they are here so the inventory is complete. The demo page is vendored rather than loaded from a CDN, so the strict content-security policy in Section 3.13 can forbid third-party origins outright.

Table 11. Components that support the demonstration, the deployment and the testing, none of which a running geocoder requires.

Component	Version	Licence	Role	Required?
MapLibre GL JS	6.10.0	BSD-3-Clause	Map rendering on the demo page	demo only
PMTiles	4.5.0	BSD-3-Clause	Single-file basemap reader, no tile server	demo only
Protomaps basemaps	5.7.2	BSD-3-Clause	Basemap style and assets	demo only
Fraunces, IBM Plex Sans Condensed, IBM Plex Mono	5.3.0	SIL OFL 1.1	Self-hosted fonts	demo only
Docker Engine and Compose	-	Apache-2.0	Packaging and process supervision	deployment convenience
Ansible	-	GPL-3.0	Provisioning the VM and the edge	deployment convenience
Caddy	-	Apache-2.0	TLS termination at the wharf	site-specific
k6	-	AGPL-3.0	The load harness in Section 2.5	testing only
pytest, Ruff	9.1.1, 0.16.8	MIT	Tests and linting	development only

2.2 Data

Both engines are loaded from the same open sources, so any difference in results is a difference in engine rather than in data. This section says what those sources are, how large they are and what was dropped.

Table 12. Data sources, raw input sizes, and what each engine holds from them. Pelias counts are Elasticsearch documents; pgeo counts are rows in its feature table.

Source	Content	Raw input	Pelias documents	pgeo features
OpenAddresses	address points (Maine statewide E911 feed)	222 MB	780,260	677,229
OpenStreetMap	addresses, streets, venues (Geofabrik extract)	87 MB	768,992	128,471
Overture Maps	places (businesses, landmarks), 2026-08-19 release	16 MB	76,582	76,582
USGS GNIS	named features: lakes, summits, streams, populated places	4 MB	20,104	20,104
Who's On First	admin polygons: towns, counties, ZIPs, neighbourhoods	5,254 MB	3,280	3,289
Census ZCTA	ZIP code centroids	1 MB	426	426
TIGER / OpenAddresses	street ranges for the Pelias interpolation service	333 MB	(not indexed)	-
Total			1,649,644	906,101

The two engines load the same inputs but store different numbers of records. The largest difference is addresses: Pelias holds **1,421,745** address documents and pgeo **702,701** (2.02 times as many in Pelias). OpenAddresses (the state's E911 address points) and OpenStreetMap both carry most Maine addresses; Pelias indexes both copies and removes duplicates when it answers a query, while pgeo removes them once at build time (same house number, street and town; OpenAddresses preferred). The data a user sees is the same; the index Pelias searches is roughly twice as large for addresses.

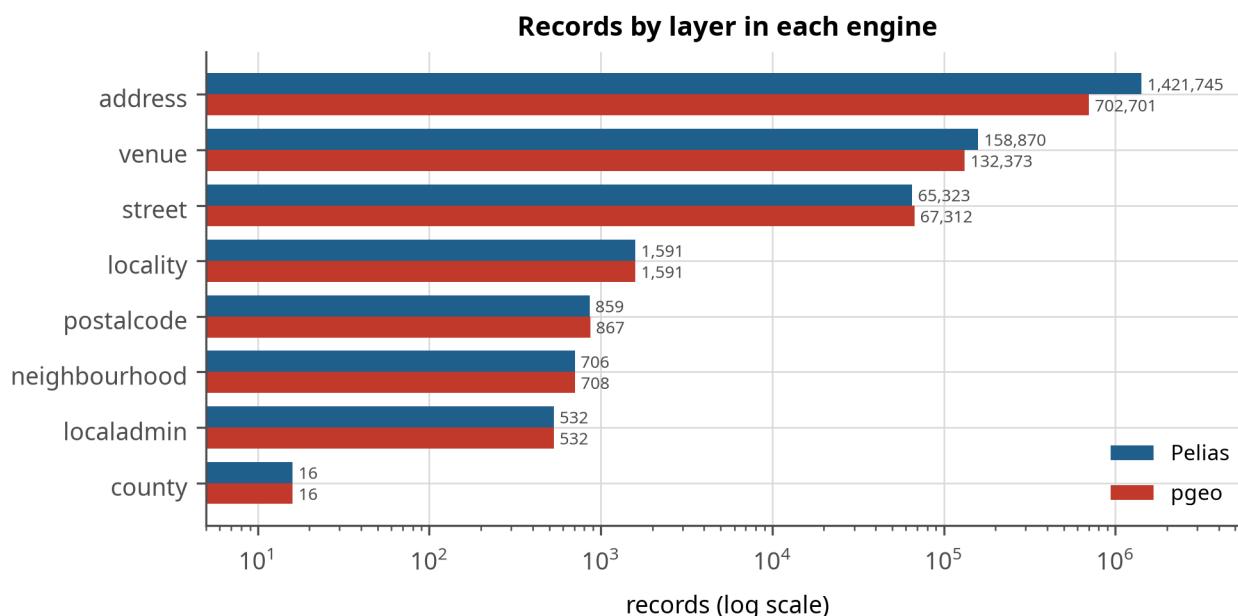


Figure 4. Records by layer in each engine (log scale). The address layer shows the build-time deduplication in pgeo; venues differ because pgeo also deduplicates OSM and Overture places that share a name and position.

Figure 4 shows where the engines hold the same data differently. The address bar is the one to read: Pelias stores 1,421,745 address documents and pgeo 702,701, because the two open sources carry most Maine addresses twice and pgeo merges them at build time while Pelias keeps both and filters at query time. The consequence is visible later in memory and disk (Section 3.5): pgeo searches a smaller store.

Table 13. Records by layer.

Layer	Pelias	pgeo	pgeo / Pelias
address	1,421,745	702,701	0.49
venue	158,870	132,373	0.83
street	65,323	67,312	1.03
locality	1,591	1,591	1.00
localadmin	532	532	1.00
neighbourhood	706	708	1.00
postalcode	859	867	1.01
county	16	16	1.00
region	1	1	1.00
country	1	0	0.00

2.2.1 Overture Maps Every Overture theme was evaluated for Maine (2026-08-19 release), 203 MB of Parquet extracts in total, of which only the 16 MB *places* file was loaded. *Places* add value and were loaded (76,582 after quality and boundary rules). *Addresses* add nothing for Maine: all 772,684 come from the national address database, which is built from the same Maine E911 address points as OpenAddresses; every apparently unique record has an OpenAddresses twin within 5 m that differs only in spelling (“78 East Grand Avenue” vs “78 E Grand Ave”), and 90% of pairs have identical coordinates. They remain a token-free fallback and a source of spelled-out street aliases, and should be re-checked for New Hampshire. *Divisions* parallel Who’s On First (1,145 localities, 916 polygons) and cannot feed Pelias’s point-in-polygon service; they were not used. *Water, land and roads* are derived from OpenStreetMap, which is already loaded.

Data problems found and fixed during preparation (all in prep/): GNIS points outside Maine, a pipe-delimited ZCTA gazetteer, an Overture extract that leaked into Portsmouth, New Hampshire, and the Pelias interpolation builder needing OpenAddresses in its legacy CSV form.

2.3 Build pipelines

How raw downloads become a searchable index or database. This matters operationally: the build is the only step that needs a large machine, and it is what a data refresh repeats.

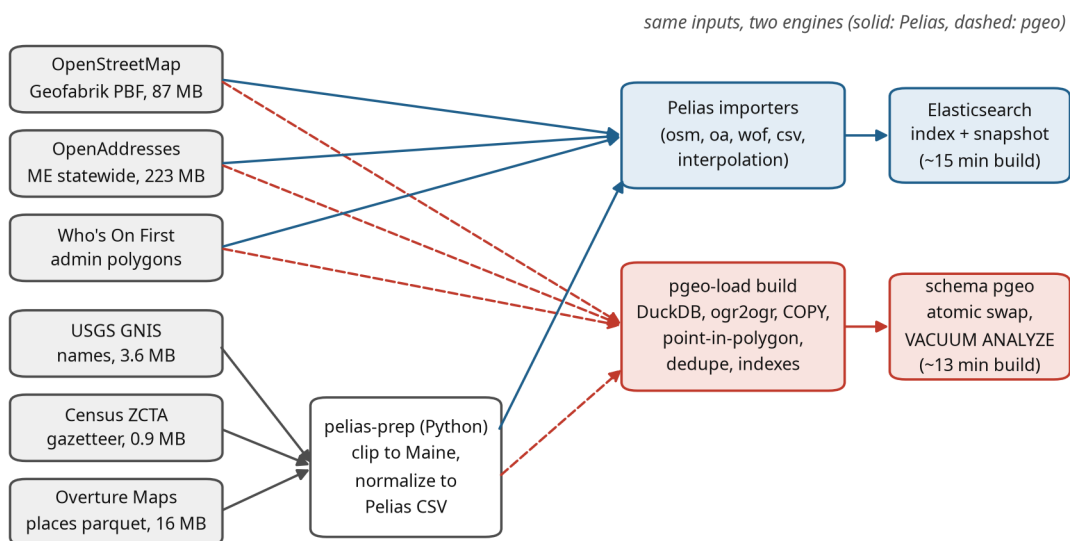


Figure 5. Data flow from the sources to each engine. Both engines read the same inputs; small sources pass through a preparation package that clips them to Maine.

Figure 5 is the reproducibility picture: both engines consume the same downloaded inputs, and everything is scripted, so a rebuild is one command and a data refresh is the same command with newer downloads (docs/REBUILD.md).

The Pelias build (OSM, OpenAddresses, Who’s On First, CSV and interpolation importers) takes about 9 minutes on the workstation once the inputs are downloaded, and produces a 435 MB index plus helper databases. The pgeo build (pgeo-load build) loads the same inputs with DuckDB[26], ogr2ogr[27] and COPY, assigns each point its admin hierarchy by point-in-polygon, deduplicates, indexes, and swaps the new schema in atomically; it took 13 min in the latest run on one core.

Table 14. pgeo build steps and their duration (latest build).

Build step	Seconds
osm	39
stage	11
enrich_index	727

Both builds, the verification suites and the deployment are scripted; `docs/REBUILD.md` in the repository is the step-by-step runbook and `scripts/rebuild_all.sh` runs it end to end (Appendix A).

A build is the one time either platform needs a large machine, and it is a machine you can rent for an hour and give back: the query host never sees the raw data or the build tools, only a 435 MB Elasticsearch snapshot or a PostgreSQL dump. Table 33 in Section 3.5 gives the measured cost of each, which is what decides whether a build can share the workstation with other work or wants a machine to itself.

2.4 Accuracy testing

The accuracy test set was built from the source data, independently of both engines: the correct answer for each case is the source record’s position (for addresses, also the house number). A case counts as correct when the first result lies within the case’s radius (for example 50 m for an address, 8 km for a town); a “miss” case (a place that does not exist in Maine) counts as correct when the engine returns nothing, or nothing with confidence of at least 0.8.

Table 15. Composition of the accuracy test set by endpoint, category and query quality.

Endpoint	Category	Exact	Typo	Variant	Miss	Total
autocomplete	Towns	75				75
autocomplete	Venues	75				75
reverse	Reverse	200				200
search	Addresses	220	87	93		400
search	Lakes, summits	83	35	32		150
search	Misses				150	150
search	Towns	92	34	24		150
search	Venues	108	42			150
search	ZIP codes	60				60
structured	Addresses	150				150

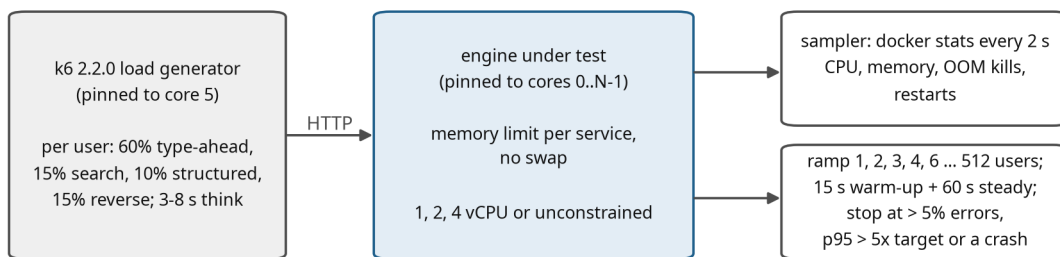
Non-unique names (there are dozens of “Mud Pond”s in Maine) were qualified with the nearest town, because a bare “Mud Pond” has no single right answer. A separate **fuzz set** of 1,800 cases corrupts 300 exact queries at six levels:

Table 16. Fuzz levels. House numbers are never corrupted, so every query stays answerable.

Level	Corruption
F0	none (baseline)
F1	one character error (swap, drop, double, keyboard neighbour)
F2	two character errors in different words
F3	three errors, abbreviation flips (St <-> Street), commas dropped, lowercase
F4	F3 plus a component dropped (town, suffix or state) and word-order changes
F5	heavy: about 25% of letters corrupted, phonetic spellings (ph -> f, ck -> k)

2.5 Load and capacity testing

How capacity was measured, and the choices that make the two engines comparable: one engine at a time, pinned to its own cores, with the same corpus, targets and ramp.



Host: AMD Ryzen 5 3600 (6 cores / 12 threads), 31 GB RAM, NVMe. The engine and the load generator never share a physical core.

Figure 6. Load-test harness. The load generator and the engine under test are pinned to different physical cores; each engine service runs with a memory limit and no swap.

Figure 6 shows why the comparison is fair: one engine at a time, pinned to its own physical cores, with the load generator on a separate core, the same corpus and the same stopping rules. Readers reproducing this on other hardware should expect different absolute numbers but the same ordering.

Each simulated user behaves like a person using the demo page: 60% of sessions type an address or place letter by letter (autocomplete requests), 15% run a full search, 10% a structured search and 15% click the map (reverse), each followed by 3 to 8 seconds of think time. Text queries are 65% exact, 12% with typos, 13% off-name variants and 10% impossible places; 10% of map clicks fall offshore.

Table 17. Load-test query corpus (drawn at random per request).

Corpus pool	Items	Used for
addresses	3,000	type-ahead and search, exact form
places	800	towns, lakes, summits
venues	1,000	businesses and landmarks
zips	200	ZIP code searches
structured	1,000	structured search field sets
typos	1,200	one or two character errors
variants	1,200	off-name variants (abbreviations, missing town)
misses	600	places that do not exist in Maine
oa_points / random_points	1,000 / 1,000	reverse geocoding on land
miss_points	300	reverse geocoding offshore (misses)
foci	120	focus points (map centers) for type-ahead

Table 18. Example requests from the load session.

Endpoint	Example request (as sent by the load generator)
Autocomplete	/v1/autocomplete?text=389 cong&size=8&focus.point.lat=43.66&focus.point.lon=-70.26
Search	/v1/search?text=389 Congress St, Portland, ME&size=8&focus.point.lat=...
Search (typo)	/v1/search?text=Moosehed Lake&size=8
Structured	/v1/search/structured?address=389 Congress St&locality=Portland®ion=ME&postalcode=04101&size=8
Reverse	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&size=6

Table 19. Latency targets (95th percentile) used as the pass criterion.

Endpoint	p95 target	Why
Autocomplete	250 ms	type-ahead must keep up with typing
Search	750 ms	a full search after pressing Enter
Structured search	750 ms	form-based search
Reverse	400 ms	map click; feels immediate
All	errors < 1%	no restarts or out-of-memory kills during a run

For every configuration the harness recreates the services with their CPU and memory limits, warms them up, runs a 3-user validation (3 minutes), then ramps through 1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384 and 512 users (15 s warm-up plus 60 s measured at each step). A step **passes** when every endpoint's p95 is within target and errors stay below 1%; the run stops when errors exceed 5%, any p95 exceeds five times its target, or a service dies.

2.5.1 Caching No response caching is configured anywhere (nginx, Caddy, the Pelias API and pgeo all answer every request afresh). The caches that do exist are warmed on purpose and are the same for both engines: containers are recreated for every configuration (clearing the Elasticsearch query cache and PostgreSQL's shared buffers), a fixed warm-up runs before measuring, and warm-up requests are excluded from the results. The operating system's page cache survives container recreation. All numbers therefore describe a server in continuous use, not a cold start. The query corpus is large (Table 17) and drawn at random, so exact repeats are rare.

2.5.2 Scope Every test in this section was run on both engines with the same session, corpus, targets and ramp; pgeo additionally with both of its front ends.

Table 20. Tested configurations. pgeo configurations were run with each front end (rest = pure SQL through PostgREST, api = FastAPI); budgets include 0.8 GB for the operating system.

Engine	Config	vCPU	Memory budget	Notes
Pelias	M0	all	unlimited	reference, 1 API worker
Pelias	C1 / C1s	1	8.3 / 9.1 GB	floor and standard memory profiles
Pelias	C2 / C3	2	8.4 / 9.5 GB	floor / standard
Pelias	C4 / C4a	4	10.8 / 10.4 GB	4 API workers / 1 API worker
pgeo	Pmin	1	1.6 GB	database 0.6 GB: smaller than the data
pgeo	P1	1	2.0 GB	database 1.0 GB
pgeo	P2	2	2.5-2.7 GB	database 1.5 GB
pgeo	P4	4	3.0-3.3 GB	database 2.0 GB
pgeo	PM	all	unlimited	reference

2.6 Compatibility testing

A contract test (`tests/compat/compat_test.py`) sends 28 documented Pelias requests to Pelias and to both pgeo front ends and checks each response for status code, envelope shape, property names, filter behaviour (results inside the requested circle, rectangle or area; no results for another country) and Pelias-shaped errors. Pelias is the reference: a check that Pelias itself fails is reported but not counted against pgeo.

2.7 Tuning procedure

pgeo was tuned in rounds. Each round started from the failing test cases (what came back, what was expected, which query step lost the right answer), changed one thing, and re-ran the full accuracy set, the fuzz set and a per-category comparison that listed every flipped case. A change was kept only if overall accuracy did not drop and no category lost more than noise.

2.7.1 Parallelism PostgreSQL uses several cores in two ways. Between queries, every connection is its own backend process, so concurrent users spread across all available cores; pgeo's connection pools are sized to about two per core, and capacity grew in proportion to the cores given (Section 3.4), which shows the cores are used. Within a single query, PostgreSQL can split large scans across parallel workers; this is switched off (`max_parallel_workers_per_gather = 0`) for serving, because pgeo's queries take 5-100 ms and read small, indexed row sets, where starting workers costs more than it saves and would take cores away

from other users. It was not measured (experiment T2 in docs/PGEO_TUNING.md). The build, which scans whole tables, does use four parallel workers.

The final state is protected by:

- **tuning profiles** (pgeo-tune): measured PostgreSQL and front-end settings per machine size, applied and verified against the running server;
- an **accuracy gate** (tests/accuracy/gate.py) that fails a build if accuracy, any category, calibration or any fuzz level regresses beyond small tolerances. Replayed against results from before the town-aware ranking step it reports five regressions, so it does catch the kind of tuning this project could otherwise lose silently;
- a **rebuild pipeline** (scripts/pgeo_rebuild.sh): build, profile, known-answer checks, gate.

3. Results

What the measurements show, in the order a reader is likely to care: accuracy first, then how each engine behaves under load, how small a machine it needs, and how the two compare feature for feature.

3.1 Accuracy

How often each engine returns the right answer, on a test set whose answers come from the source data rather than from either engine. Accuracy is the axis where the two differ most, and the one a user notices first.

Table 21. Accuracy on 1,560 cases. Correct = right answer at rank 1 (misses: nothing, or low confidence); Hit@5 = right answer in the top five; confidence columns are the mean confidence of the first result when right and when wrong.

Engine	Correct	Hit@1	Hit@5	Exact	Typo	Variant	Miss	Conf. right / wrong	p50 ms
Pelias	75.5%	77.0%	83.5%	88%	30%	85%	41%	0.98 / 0.87	13
pgeo pure SQL	95.8%	94.4%	98.2%	98%	87%	95%	95%	0.92 / 0.69	39
pgeo FastAPI (rule parser)	95.8%	94.4%	98.2%	98%	87%	95%	95%	0.92 / 0.69	46
pgeo FastAPI (libpostal service)	94.0%	92.4%	96.9%	95%	87%	93%	94%	0.92 / 0.60	45

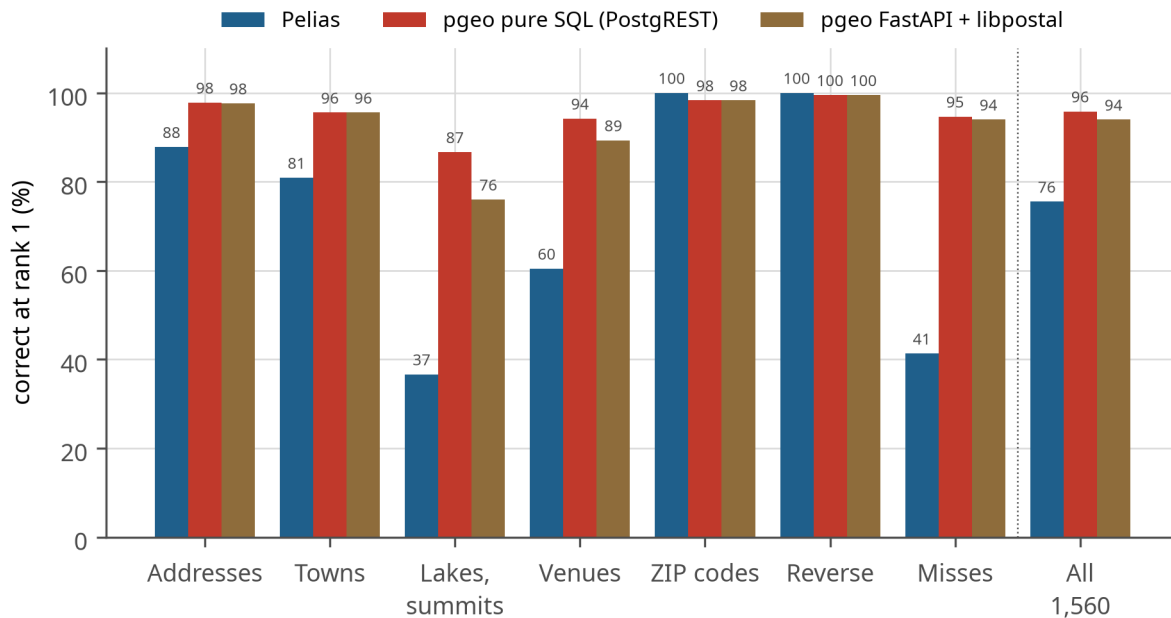


Figure 7. Correct answers by category. pgeo leads in every category except ZIP codes and reverse geocoding, where both engines are near perfect.

Table 22. Accuracy by category (all query qualities).

Category	Cases	Pelias	pgeo pure SQL	pgeo FastAPI (rule parser)	pgeo FastAPI (libpostal service)
Addresses	550	88%	98%	98%	98%
Towns	225	81%	96%	96%	96%
Lakes, summits	150	37%	87%	87%	76%
Venues	225	60%	94%	94%	89%
ZIP codes	60	100%	98%	98%	98%
Reverse	200	100%	100%	100%	100%

Category	Cases	Pelias	pgeo pure SQL	pgeo FastAPI (rule parser)	pgeo FastAPI (libpostal service)
Misses	150	41%	95%	95%	94%

The categories where pgeo gains most are the ones where the query and the data use different words for the same place. Pelias ranks text with Elasticsearch's term matching over analysed tokens; a misspelled token matches nothing, and its admin lookup (placeholder) needs exact town names. pgeo compares whole strings by trigram similarity and treats the queried town as a **location** rather than a name to match exactly: "Calvary Bible Church, Stratton" is found although the church lies in Eustis, because it is near Stratton. That single change raised venue accuracy from 81% to 94%.

What this means

Two thirds of pgeo's advantage is on input people actually type: misspellings, half-remembered venue names and places that do not exist. In a dispatch application this is the difference between an address that resolves on the first try and one an operator has to correct by hand: on this test set Pelias leaves 382 of 1,560 queries wrong or unanswered, pgeo 65.

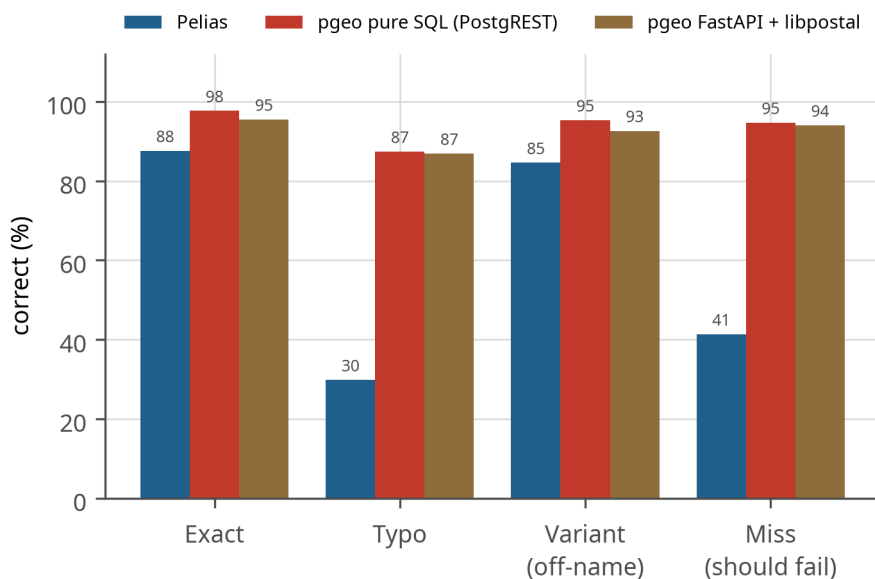


Figure 8. Accuracy by query quality. Typos separate the engines most; "miss" means the engine correctly returned nothing or only low-confidence results for a place that does not exist.

Figure 8 separates the engines by input quality. On clean input both are strong (88% and 98%); the gap opens on the input people actually type. Typos are the extreme case: 30% against 87%.

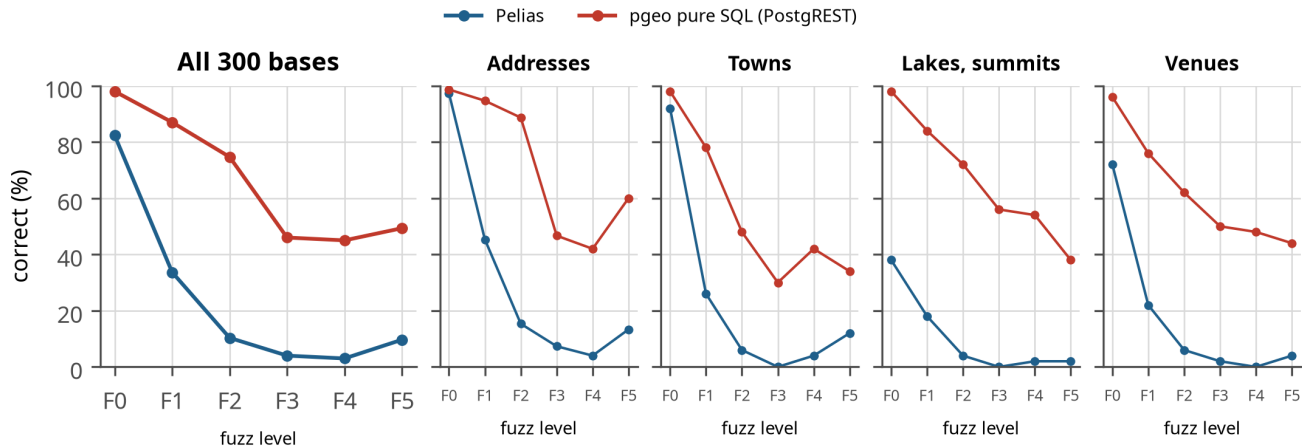


Figure 9. Accuracy as queries are progressively corrupted (fuzz levels F0-F5), overall and by category. Pelias falls to near zero from two character errors; pgeo degrades gradually.

Table 23. Fuzz results (correct, %).

Engine	F0	F1	F2	F3	F4	F5
Pelias	82%	34%	10%	4%	3%	10%
pgeo pure SQL	98%	87%	75%	46%	45%	49%

3.1.1 Calibration A confidence score is useful only if wrong answers get lower scores than right ones. pgeo’s first-result confidence averages 0.92 when right and 0.69 when wrong; Pelias scores 0.98 and 0.87, so a client can use pgeo’s confidence to decide when to ask the user.

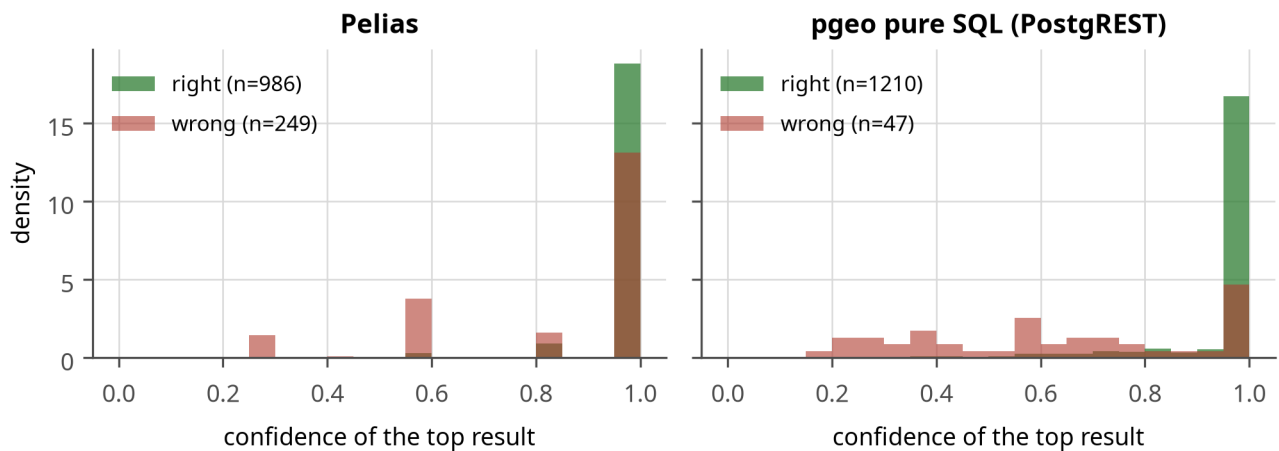


Figure 10. Distribution of first-result confidence for right and wrong answers (misses excluded). pgeo separates the two groups more clearly.

Figure 10 is about trust rather than accuracy: it shows how well each engine’s confidence separates its right answers from its wrong ones. Pelias reports high confidence for both (0.98 and 0.87), so a client cannot use the number to decide anything. pgeo’s wrong answers cluster low (0.69 on average against 0.92 when right), so an application such as LANCER can accept matches above a threshold and queue the rest for a human.

3.1.2 Remaining failures 65 of 1,560 cases still fail on pgeo (Table 24). Eight “misses” are test-set errors (Death Valley in Minot and “The Eiffel Tower of Paris, Maine” exist); about ten are genuinely ambiguous (chain branches, two towns with the same name); short autocomplete prefixes without a focus point (“Nor”, “Tim Ho”) cannot be resolved by any engine; the hardest remaining class is typos in both the place name and the town.

Table 24. Remaining pgeo failures by category.

Category (query quality)	Failures
Lakes, summits (typo)	14
Venues (exact)	11
Misses (miss)	8
Addresses (typo)	5
Towns (exact)	5
Addresses (variant)	4
Towns (typo)	4
Lakes, summits (exact)	4
Addresses (exact)	3
Lakes, summits (variant)	2
Venues (typo)	2
Reverse (exact)	1
Towns (variant)	1
ZIP codes (exact)	1

3.2 Tuning trajectory

How pgeo got from its first working version to its current accuracy, one measured change at a time. This is included because it shows which ideas paid off and which did not, and because it is the part of the result most at risk of being fitted to the test set.

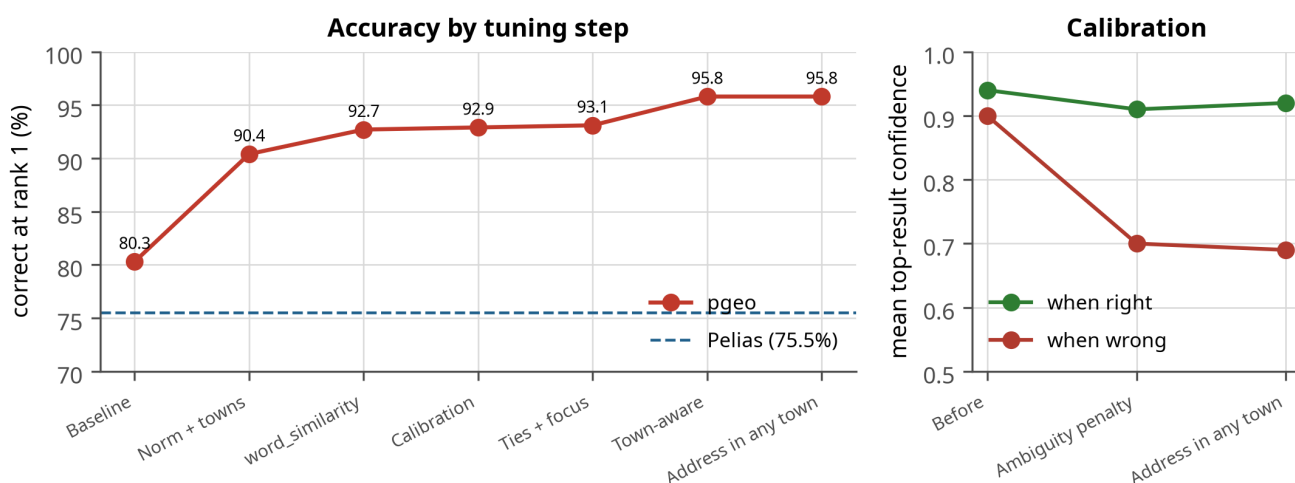


Figure 11. Left: pgeo accuracy after each tuning step, with Pelias for reference. **Right:** first-result confidence when right and when wrong; the ambiguity penalty widened the gap.

Figure 11 is the tuning history: nine rounds took pgeo from 80.3% to 95.8%. Two things stand out. Most of the gain came from understanding how people write place names (abbreviations, the town as a location,

ties), not from more powerful matching. And the calibration panel shows the ambiguity penalty pulling wrong-answer confidence down from 0.90 to 0.69 while right answers barely moved (0.94 to 0.92), which is what makes the confidence score usable.

Two failures behind these steps are worth naming, because they are the kind a test set catches and a demonstration does not. “Portland, ME” returned venues literally named “Portland, ME” rather than the city, which is why similarity is measured in the direction (query, name). And in autocomplete “st” expanded to “saint” as readily as to “street”, so a half-typed street query returned saints’ places until the expansion became context-dependent. One caveat belongs with the trajectory: the word_similarity round also corrected the test set (non-unique names such as “Mud Pond” were qualified with their town, for both engines), so the step before it and the step after it are not measured on an identical set.

Key result

One missing database index (the join from admin polygons to their features) made every reverse request scan all 906,101 features. Adding it cut reverse latency from 225-393 ms to 5-18 ms and raised the number of users one vCPU can serve from 4 to 24, six times as many, from a single line of DDL – found only because the load tests measured each endpoint separately.

Table 25. Tuning changes, the problem each solved, and the measured effect.

Step	Change	Problem solved	Effect
Normalization	Abbreviations expanded (rd -> road, st -> street or saint)	typos in spelled-out words failed (similarity 0.25 vs 0.81)	part of 80.3% -> 90.4%
Towns	towns matched on WOF and postal city; ZIP only a fallback	structured search 0% at rank 1 (ZIP outranked the address)	structured 0% -> 100%
Dedupe	OA/OSM addresses merged; results one per label	duplicated towns and addresses	clean lists
word_similarity	direction (query, name)	“Mountain” matched every query containing the word	lakes/summits 59% -> 77%
Calibration	confidence divided among tied distinct places	wrong answers were mostly ties (70 of 85)	confidence wrong 0.90 -> 0.70
Ties + focus	candidates ordered by distance to the focus before the row limits	hundreds of “Main Street”s cut to 40 before the focus applied	“main st” near Bangor returns Bangor
Town-aware	the queried town is a location (distance-based agreement)	village or nearest town given instead of the containing one	venues 81% -> 94%; 93.1% -> 95.8%
Address in any town	house number looked up on the best street names statewide	25 of 150 “Church Street” towns kept arbitrarily	confidence wrong 0.73 -> 0.69
Reverse index	index on the admin join	every reverse request scanned all features	see Section 3.4

3.3 Latency at the 3-user target

This section answers the question the project was commissioned to answer – is either engine fast enough for three people? – and the answer turns out not to separate them, which is why every later section is about load, memory and accuracy instead.

The configurations named in this section and the next are:

- **C1, C2, C4** – Pelias on one, two and four vCPU; **C1s, C3, C4a** are variants of those with a different memory profile or worker count; **M0** is Pelias unconstrained.

- **Pmin, P1, P2, P4** – pgeo on one, one, two and four vCPU with budgets from 1.6 to 3.3 GB; **PM** is pgeo unconstrained.
- **E1, E2, E4** – pgeo given the *same memory as Pelias* at each CPU size, for the equal-resources comparison in Section 3.4.
- Each pgeo configuration is prefixed by its front end: **rest-** is the pure-SQL path through PostgREST, **api-** is FastAPI with the rule parser, and **api-svc-** is FastAPI with the libpostal service. So api-P4 is FastAPI on four vCPU.

Full details of every configuration are in Table 20.

Table 26. Median and p95 latency at 3 users (ms).

Config	Autocomplete p50 / p95 ms	Search p50 / p95 ms	Structured p50 / p95 ms	Reverse p50 / p95 ms
C1	15 / 37	26 / 50	28 / 40	24 / 40
C2	9 / 22	20 / 36	19 / 27	19 / 26
C3	8 / 19	14 / 27	13 / 20	10 / 39
C4	8 / 23	21 / 30	17 / 22	22 / 25
M0	8 / 23	20 / 27	20 / 25	20 / 25
rest-Pmin	19 / 52	42 / 114	81 / 120	13 / 18
rest-P1	19 / 59	44 / 111	60 / 100	5 / 12
rest-P2	15 / 44	30 / 97	88 / 105	4 / 5
rest-P4	15 / 43	43 / 101	91 / 98	4 / 6
rest-PM	17 / 46	28 / 100	42 / 85	5 / 6
api-Pmin	17 / 62	62 / 112	73 / 117	13 / 15
api-P1	18 / 59	39 / 109	103 / 127	11 / 14
api-P2	15 / 54	34 / 95	91 / 106	5 / 14
api-P4	18 / 49	41 / 108	108 / 116	4 / 15
api-PM	17 / 57	50 / 99	92 / 104	4 / 15
api-svc-P2	14 / 47	41 / 101	48 / 93	4 / 16
api-svc-P4	17 / 51	45 / 111	93 / 101	5 / 14

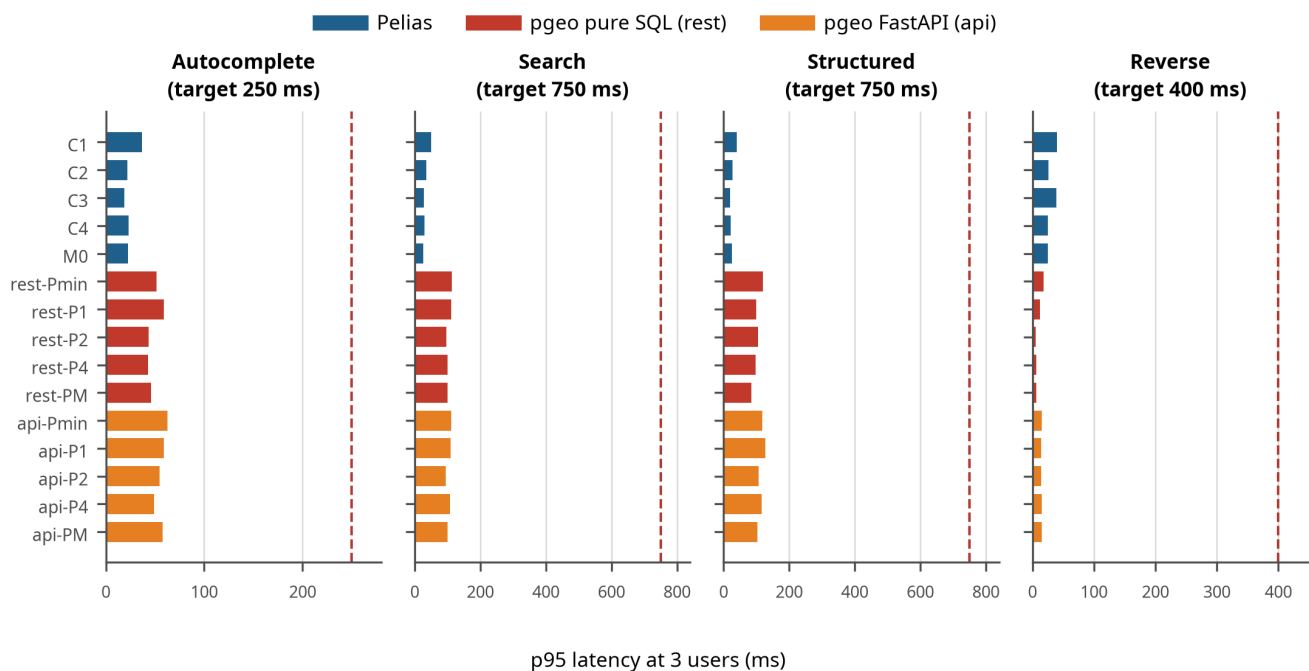


Figure 12. p95 latency per endpoint at 3 users for each configuration, against its target (dashed). Every configuration of both engines meets every target.

Figure 12 answers the question this project started with: at the 3-user target every configuration of both engines, down to the smallest, sits well inside every latency target. Nothing in the 3-user scenario forces a choice between the engines; the differences appear only under heavier load or in accuracy.

At 3 users every configuration of both engines meets every target with a wide margin, so the 3-user goal is not what separates them. Pelias answers in 20-50 ms at the 95th percentile on every endpoint: Elasticsearch resolves text and geography in compiled index lookups. pgeo's autocomplete takes about 40 ms and its search and structured search about 100 ms, because a forward search runs several candidate queries and a scoring step in PL/pgSQL inside one database backend. Reverse geocoding was pgeo's slow endpoint (225-393 ms, close to its 400 ms target on one vCPU) until the missing admin-join index was added; it now takes 5-18 ms (Table 29).

3.4 Capacity under load

How many concurrent users each configuration serves before missing a latency target, and what gives out first. This is the axis where Pelias leads, and the section is where the comparison is made at equal resources as well as at equal cost.

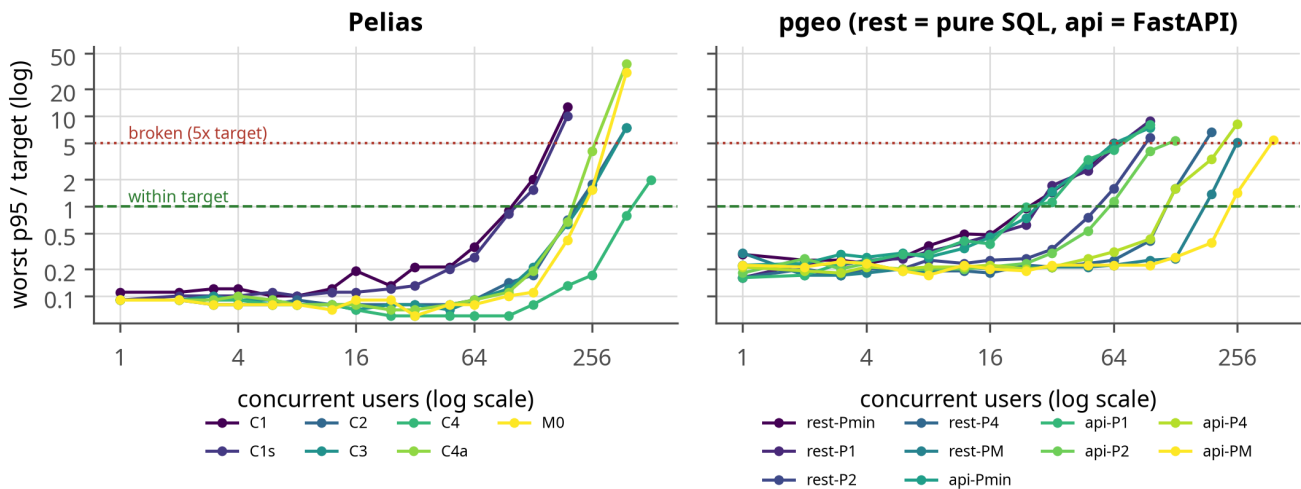


Figure 13. Worst endpoint p95 relative to its target as users are added. Above 1 the configuration misses a target; above 5 the run stops. At 1-4 users some endpoints receive only a few requests per window, so single points are noisy.

Figure 13 shows how each configuration degrades as users are added: flat while there is headroom, then rising steeply once the CPUs saturate. The practical consequence is that these services give little warning: a configuration comfortably inside its targets at 24 users can be three times over them at 48. Capacity planning should therefore leave a factor of two, not ten percent.

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
--------	------	--------	----------	------------------	-----------	-------------------	----------------	-------------------

Table 27. Pelias capacity by configuration.

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
C1	1	8.3 GB	heap 768m, 1 worker(s)	96	192	autocom- plete, search, reverse at 128	99	5.9 GB
C1s	1	9.1 GB	heap 768m, 1 worker(s)	96	192	autocom- plete, reverse at 128	102	5.8 GB
C2	2	8.4 GB	heap 768m, 2 worker(s)	192	384	autocom- plete, reverse at 256	209	6.1 GB
C3	2	9.5 GB	heap 1g, 2 worker(s)	192	384	autocom- plete, reverse at 256	213	6.3 GB
C4	4	10.8 GB	heap 2g, 4 worker(s)	384	not reached	autocom- plete, search, structured, reverse at 512	403	7.5 GB
C4a	4	10.4 GB	heap 2g, 1 worker(s)	192	384	autocom- plete, search, structured, reverse at 256	194	7.1 GB
M0	all	-	heap 4g, 1 worker(s)	192	384	autocom- plete, reverse at 256	215	9.3 GB

Table 28. pgeo capacity by configuration (after the reverse-geocoding index fix).

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
rest-Pmin	1	1.6 GB	sb 128MB, 4 conn.	24	64	autocom- plete at 32	24	0.3 GB
rest-P1	1	2.0 GB	sb 256MB, 4 conn.	24	96	autocom- plete at 32	25	0.4 GB
rest-P2	2	2.5 GB	sb 384MB, 6 conn.	48	96	autocom- plete, reverse at 64	53	0.5 GB

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
rest-P4	4	3.0 GB	sb 512MB, 10 conn.	96	192	autocom- plete at 128	108	0.6 GB
rest-PM	all	-	sb 1GB, 16 conn.	128	256	autocom- plete at 192	144	0.6 GB
api-Pmin	1	1.6 GB	sb 128MB, 4 conn.	24	96	autocom- plete at 32	26	0.3 GB
api-P1	1	2.0 GB	sb 256MB, 4 conn.	24	96	autocom- plete at 32	27	0.4 GB
api-P2	2	2.7 GB	sb 384MB, 6 conn.	48	128	autocom- plete at 64	52	0.7 GB
api-P4	4	3.3 GB	sb 512MB, 10 conn.	96	256	autocom- plete, reverse at 128	103	0.8 GB
api-PM	all	-	sb 1GB, 16 conn.	192	384	autocom- plete at 256	213	0.8 GB
api-svc-P2	2	4.7 GB	sb 384MB, 6 conn.	48	128	autocom- plete at 64	52	2.5 GB
api-svc-P4	4	5.3 GB	sb 512MB, 10 conn.	96	192	autocom- plete at 128	106	2.8 GB

Table 29. Effect of the reverse-geocoding index on pgeo capacity and reverse latency.

Config	Users within SLO, before	after	Reverse p95 at 3 users (ms), before	after
rest-Pmin	4	24	393	18
rest-P1	4	24	348	12
rest-P2	24	48	226	5
rest-P4	64	96	225	6
rest-PM	128	128	255	6
api-Pmin	8	24	325	15
api-P1	6	24	340	14
api-P2	32	48	291	14
api-P4	96	96	224	15
api-PM	192	192	257	15

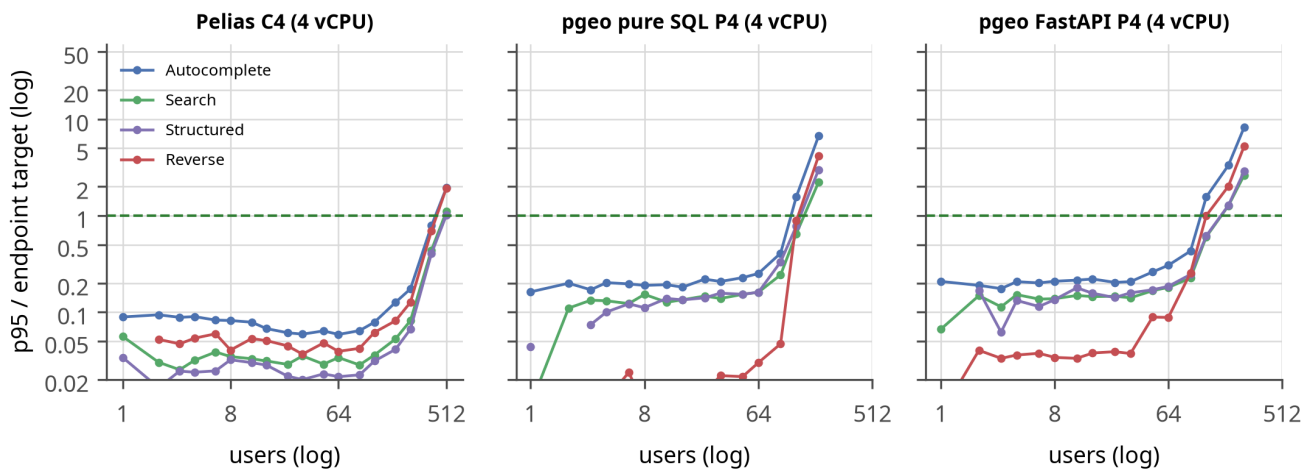
**Figure 14. p95 per endpoint relative to its target on 4 vCPU.** The first line to cross 1 is the endpoint that limits capacity.

Figure 14 shows which endpoint fails first, which is where tuning effort belongs. On pgeo it is autocomplete, because it has the tightest target (250 ms) and fires several times per typed word; on Pelias the four endpoints rise together because they share one Elasticsearch.

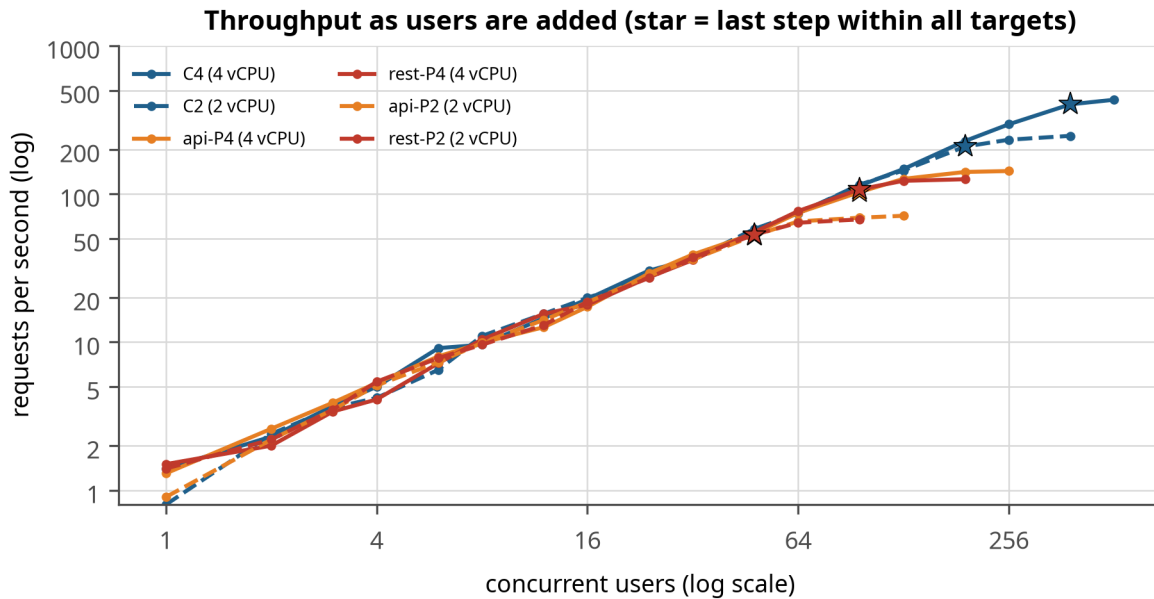


Figure 15. Throughput as users are added; the star marks the last step within all targets.

Figure 15 shows throughput still climbing after the latency targets are missed (the stars): both engines keep answering, just too slowly. An operator watching only requests per second would not notice the service becoming unpleasant, which is why the targets, not throughput, define capacity here.

3.4.1 The same memory for both engines Every comparison so far gave pgeo a smaller budget than Pelias, because that is what each engine needs. It is a fair way to compare cost and an unfair way to compare engines: a reader is entitled to ask whether Pelias only leads because it was handed five times the memory. So pgeo was re-run at exactly Pelias's budget – 8.3, 8.4 and 10.8 GB at one, two and four vCPU (configurations E1, E2 and E4).

Table 30. The same benchmark with the memory budgets equalised. pgeo holds the same number of users at Pelias's budget as at its own, so the throughput difference is not a memory artefact.

CPU	pgeo, its own budget	pgeo, Pelias's budget	Pelias
1 vCPU	24 (2.0 GB)	24 (8.3 GB)	96 (8.3 GB)
2 vCPU	48 (2.5 GB)	48 (8.4 GB)	192 (8.4 GB)
4 vCPU	96 (3.0 GB)	96 (10.8 GB)	384 (10.8 GB)

pgeo answers exactly the same number of users at 8.3 GB as at 1.6 GB. Not approximately – the same number, at every CPU size. The reason is in Figure 16: the whole database is 847 MB, so at 1.6 GB it already had room for the entire working set, and the extra 6.7 GB has nothing to cache that was not cached before.

Key result

The four-to-one throughput gap is a CPU difference, not a memory difference. Giving pgeo Pelias's full memory budget changes its capacity by nothing at all. That also means the earlier comparisons were not flattering pgeo by starving Pelias: the two engines were already being compared at the only resource that binds either of them.

This cuts both ways, and the second direction matters more for an operator. Pelias cannot give its memory back – its floor is 7.17 GB because five services load fixed data before serving anything (Section 3.12) –

whereas pgeo cannot use the memory if you give it. So the fair statement is not “pgeo is cheaper” or “Pelias is faster” but this: **at equal memory Pelias serves four times the users; at equal throughput pgeo needs a fifth of the memory; and neither engine can trade one for the other.**

3.4.2 How each engine scales Both engines are CPU-bound under this load, and both scale close to linearly with vCPUs as long as their workers can use them: pgeo holds about 24 users per vCPU (24, 48 and 96 users on 1, 2 and 4 vCPU) and Pelias about 96 (96, 192 and 384). The ratio of four is the difference in CPU cost per request: compiled index lookups in Elasticsearch against interpreted candidate queries and scoring in PL/pgSQL.

What this means

Plan capacity as about 24 concurrent users per vCPU for pgeo and about 96 for Pelias, then halve it for headroom: both engines go from comfortable to three times over target within one doubling of load.

3.4.3 What gives out first After the index fix, autocomplete is the first endpoint over its target on every pgeo configuration (Figure 14): it has the tightest target (250 ms) and runs most often (60% of sessions type ahead, several requests per word). On Pelias the endpoints degrade more evenly (which one crosses first varies by configuration), because they share Elasticsearch.

3.4.4 Memory is not pgeo’s limit The configuration whose database memory (0.6 GB) is smaller than the data holds as many users as the one with 1.0 GB (24 and 24): the working set of hot index pages is far smaller than the 847 MB of tables and indexes. Pelias, in contrast, has a memory floor set by its services (libpostal’s model, the interpolation database, placeholder’s in-memory tables) regardless of load.

3.4.5 Front end PostgREST and FastAPI hold the same number of users up to 4 vCPU. Unconstrained, FastAPI reached 192 users and PostgREST 128: only at high concurrency does the gateway’s per-request overhead show.

Pelias spreads work across services in different languages: Elasticsearch does the text and geo matching in compiled code, and the Node.js API adds parsing and ranking. With four API workers on 4 vCPU (C4) it reaches 384 users; with one worker on the same CPUs (C4a) only 192, because a single Node.js worker becomes the bottleneck.

3.5 Resources

What each platform needs to run and to build. The two are very different numbers, and conflating them is the most common way to size a geocoder wrongly.

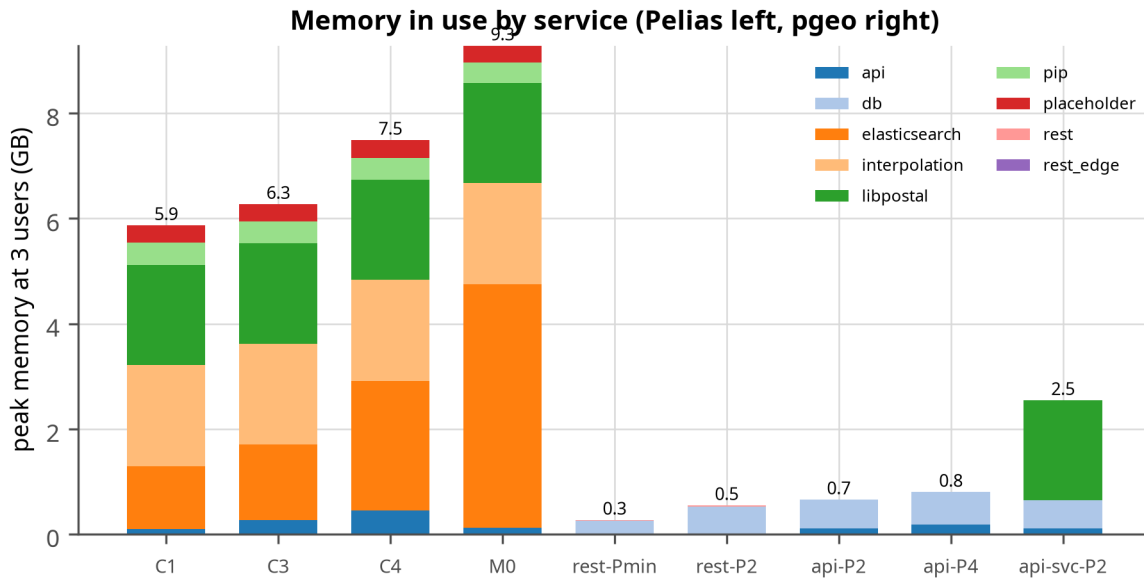


Figure 16. Peak memory in use by service at 3 users. Pelias carries the libpostal model (~2 GB), the interpolation database and Elasticsearch’s heap regardless of load; pgeo’s memory is mostly the PostgreSQL page cache for a 847 MB database.

Figure 16 explains the memory gap in one picture. Pelias pays for the libpostal model, the interpolation database and an Elasticsearch heap before it serves a single request; pgeo’s memory is mostly page cache for a 847 MB database, which is why it still works when the budget is cut to a fraction. This is the difference between a \$5 and a \$48 server (Section 3.5).

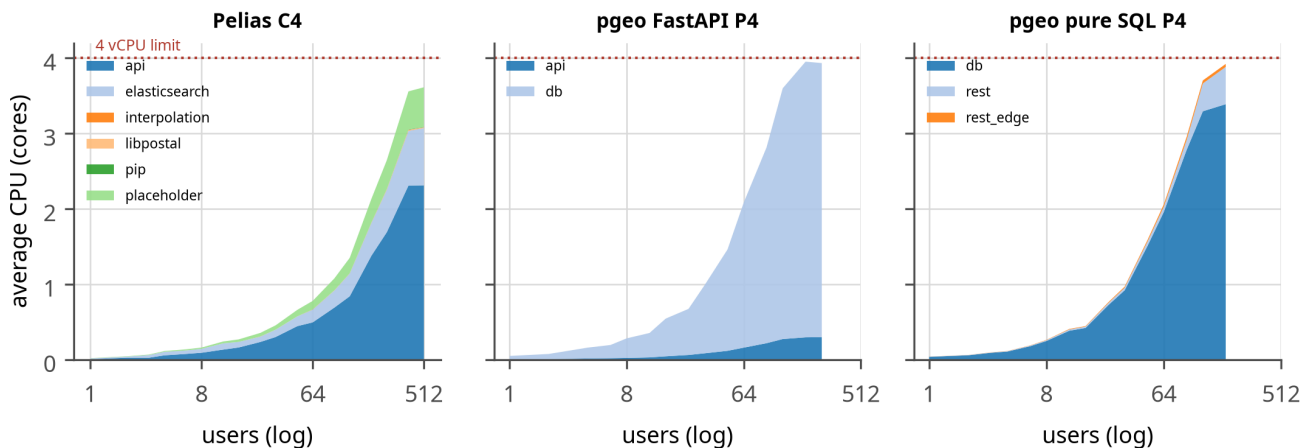


Figure 17. Average CPU by service as users are added on 4 vCPU.

Figure 17 shows where the CPU goes. In pgeo almost all of it is the database process doing the matching; in Pelias it is split between Elasticsearch and the Node.js API, and the API becomes the bottleneck when it runs with too few workers (configuration C4a reached only 192 users against 384 for C4 with the same CPUs). Anyone deploying Pelias should set one API worker per vCPU.

Resource requirements

3.5.1 Operation Table 31 turns the capacity results into a sizing guide: for a target number of concurrent users, the smallest tested configuration of each engine that kept every endpoint within its latency target.

Budgets include 0.8 GB for the operating system. One “user” is a person actively using the search page (a request every 3 to 8 seconds); a service with many occasional users needs far fewer concurrent slots than it has users.

Table 31. Sizing guide: smallest tested configuration that meets all latency targets for a given number of concurrent users. Only configurations with a memory budget are considered; “not reached in tests” means not reached within the tested budgets (the unconstrained reference runs M0 and PM appear in the capacity tables).

Concurrent users	Pelias	pgeo pure SQL	pgeo FastAPI
3	1 vCPU, 8.3 GB (C1)	1 vCPU, 1.6 GB (rest-Pmin)	1 vCPU, 1.6 GB (api-Pmin)
10	1 vCPU, 8.3 GB (C1)	1 vCPU, 1.6 GB (rest-Pmin)	1 vCPU, 1.6 GB (api-Pmin)
25	1 vCPU, 8.3 GB (C1)	2 vCPU, 2.5 GB (rest-P2)	2 vCPU, 2.7 GB (api-P2)
50	1 vCPU, 8.3 GB (C1)	4 vCPU, 3.0 GB (rest-P4)	4 vCPU, 3.3 GB (api-P4)
100	2 vCPU, 8.4 GB (C2)	not reached in tests	not reached in tests
200	4 vCPU, 10.8 GB (C4)	not reached in tests	not reached in tests
400	not reached in tests	not reached in tests	not reached in tests

Table 32. Operating requirements of each platform (Maine data).

Resource	Pelias	pgeo
Containers	6 (Elasticsearch, API, libpostal, placeholder, pip, interpolation)	2 (PostgreSQL, PostgREST) or 2 (PostgreSQL, FastAPI)
Memory in the standard profile	8.3 GB: libpostal model ~2 GB, interpolation ~2 GB, Elasticsearch heap 0.8-2 GB. Section 3.12 shrinks each service to find the real floor	1.6 GB (database 0.6 GB); floor in Section 3.12
Memory that grows with load	placeholder, OOM-killed at 0.4-0.5 GB once load arrives (at 16 users on one vCPU, 12 on two, 128 on the four-vCPU configurations); 0.8 GB needed. pip is OOM-killed at 0.4 GB and the interpolation service crash-loops below 1.9 GB	PostgreSQL backends (a few MB each), page cache
Disk (Maine)	~1.4 GB Elasticsearch data plus ~0.9 GB helper databases; 435 MB snapshot to ship	654 MB of tables and indexes; data directory ~2.6 GB, the difference being write-ahead log (max_wal_size 4 GB) and space still held by tables the build replaced; dump to ship. Size the disk for three to four times the database
CPU scaling	add API workers with CPUs (1 worker on 4 vCPU halves capacity)	add connections with CPUs; front end workers for FastAPI
Scaling out	Elasticsearch replicas and more API instances	read replicas behind PostgREST
Updating data	rebuild index on a workstation, ship snapshot, restore	rebuild database on a workstation, ship dump, restore

What this means

The memory floors put the two platforms in different price classes: pgeo fits the cheapest shared-CPU plans most providers sell, while Pelias needs a mid-range server for the same three users. At advertised

rates that is about \$60 a year on a 1 GB plan, or \$145 on a 2 GB plan with room to spare, against about \$580 for the 8 GB plan Pelias needs. The 1 GB plan is confirmed on a real machine in Section 3.12, so ten times is the honest figure.

3.5.2 Build Both engines are built on a workstation and shipped to the query host, so the query host never needs the raw data or the build tools. Table 33 gives the measured cost of a full build of each engine.

Table 33. Build resources: a full build of each engine, measured with `tests/build/measure_build.py` (wall time, peak memory of all build containers and processes, average and peak CPU, disk used by the result).

Engine	Wall time	Peak memory	CPU cores (avg / peak)	Disk after build	Measured
Pelias	9 min	11.5 GB	2.9 / 8.1	8.0 GB	2026-09-19
pgeo	13 min	5.5 GB	1.2 / 4.5	2.5 GB	2026-09-19

Caution

pgeo serves three users in 1.40 GB but is built in 5.5 GB, four times its own runtime floor. The cheap VPS the floor analysis points at can run the geocoder; it cannot build it. Both platforms are built on a workstation and shipped as a snapshot or a dump, and that is a requirement rather than a convenience.

The two builds fail the “which is cheaper” question in opposite directions, which is worth noting because it cuts against the rest of this report. Pelias builds *faster* – 9 minutes against 13 – because its importers run as separate processes across several cores, where pgeo’s loader is one Python process feeding one database. pgeo is the cheaper build in every other respect: half the peak memory (5.5 GB against 11.5), 1.2 cores on average against 2.9, and 2.5 GB of result on disk against 8.0. An operator with a spare machine and no patience should prefer Pelias’s build; an operator renting one by the hour should prefer pgeo’s.

What this costs on a shared-CPU VPS The floors in Section 3.12 are small enough to matter commercially: the difference between the two platforms is the difference between the cheapest tier a provider sells and a mid-range server. Advertised shared-CPU plans (checked 2026-09-19; verify before ordering, and note that Hetzner raised cloud prices in June 2026):

Table 34. Advertised shared-CPU VPS plans and whether each platform’s measured floor fits. Sources: [Linode/Akamai](#), [DigitalOcean](#), [Vultr](#), [Hetzner](#), [AWS Lightsail](#), [Oracle free tier](#).

Provider and plan	vCPU (shared)	RAM	Disk	Price per month	Fits pgeo	Fits Pelias
DigitalOcean Basic 512 MB	1	0.5 GB	10 GB	\$4	no	no
Vultr Cloud Compute 512 MB (IPv4)	1	0.5 GB	10 GB	\$3.50	no	no
AWS Lightsail nano (IPv6 only)	2	0.5 GB	20 GB	\$3.50	no	no
Linode (Akamai) Nanode 1 GB	1	1 GB	25 GB	\$5	tight: see Section 3.12	no
Vultr Cloud Compute 1 GB	1	1 GB	25 GB	\$5	tight: see Section 3.12	no

Provider and plan	vCPU (shared)	RAM	Disk	Price per month	Fits pgeo	Fits Pelias
DigitalOcean Basic 1 GB	1	1 GB	25 GB	\$6	tight: see Section 3.12	no
DigitalOcean Basic 2 GB / Linode 2 GB	1	2 GB	50 GB	\$12	yes	no
Hetzner CX22	2	4 GB	40 GB	EUR 3.79	yes, with room	no
Hetzner CAX11 (Arm)	2	4 GB	40 GB	EUR 5.99	yes, with room (Arm images needed)	no
Linode 8 GB / DigitalOcean 8 GB	4	8 GB	160 GB	\$48	yes	tight: see Section 3.12
Oracle Cloud Always Free (Arm)	2 OCPU	12 GB	200 GB	free tier (halved in June 2026)	yes	yes

Measured against these plans: pgeo's 1.40 GB floor is 0.60 GB of containers plus the 0.8 GB this study reserves for the operating system, so a 1 GB plan works only with a minimal OS and nothing else resident, while a 2 GB plan leaves real headroom; Pelias's 7.17 GB floor needs the 8 GB tier, four to ten times the monthly price for the same three users: the 1 GB plan is confirmed below, so the gap is the full ten times.

The 1 GB row was confirmed on a real machine rather than inferred: a temporary 1 vCPU / 1 GB VM ran the three-user load with the worst endpoint at 21% of its target and held 32 concurrent users (Section 3.12). It needs one caveat, though. `pgeo-tune auto --cpus 1 --mem-gb 1` refuses to produce a configuration: its sizing rule reserves 0.8 GB for the operating system and 0.35 GB for the HTTP front end, which leaves the database less than nothing. The settings for that VM were written by hand. A 1 GB plan therefore works, and is the cheapest way to run this geocoder, but it is a configuration this project has measured rather than one its tooling will size for you.

3.5.3 Data volume For Pelias, capacity fell from 384 users with admin areas only to 192 with every source (Section 3.6); the index grew to 435 MB. pgeo's database grows with the same sources to 654 MB, and its capacity falls from 192 users to 48 across the same subsets (Section 3.6.1) – a four-fold loss where Pelias loses two. Doubling the data (for example adding New Hampshire) roughly doubles both stores, but not the cost per query: what both engines pay for is the number of names a text query can plausibly match, and pgeo pays about twice as much per name as Pelias does.

3.6 Data volume

Whether the results survive more data. A one-state geocoder that collapses at two states would be a curiosity rather than a tool, so capacity was re-measured against progressively larger datasets.

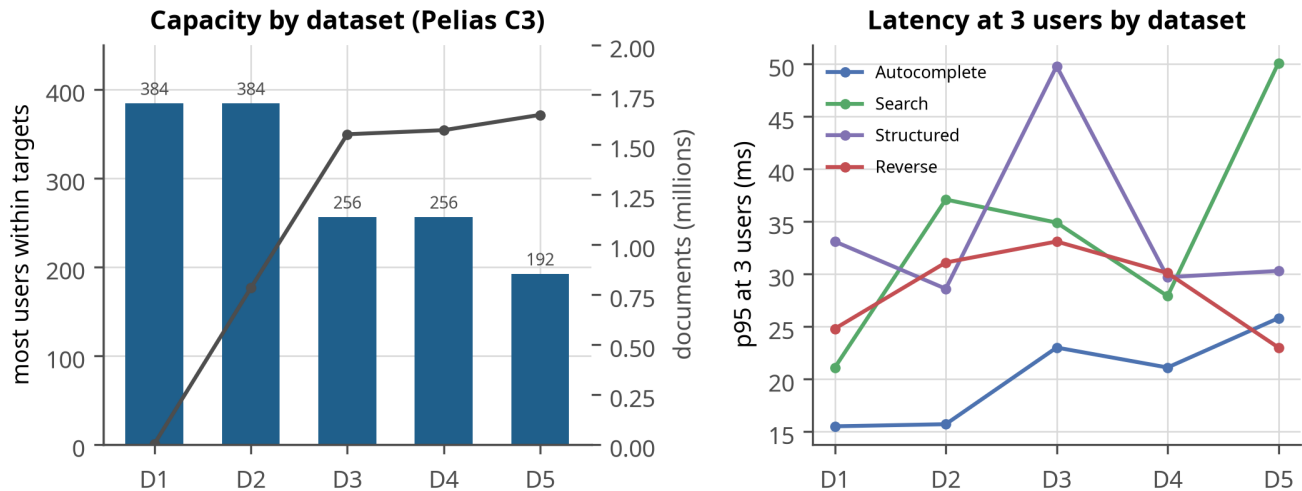


Figure 18. Pelias capacity (bars) and document count (line) for cumulative datasets D1-D5 at fixed resources (2 vCPU, 9.5 GB), and p95 latency at 3 users.

Figure 18 separates data volume from data kind. Adding 780,000 address documents cost nothing measurable, while adding OpenStreetMap and Overture places each cost a ramp step: what matters is not how many records exist but how many of them a text query can plausibly match. Adding New Hampshire should therefore be judged by how many new names it brings, not by gigabytes.

Table 35. Pelias by dataset: D1 Who's On First; D2 + OpenAddresses; D3 + OpenStreetMap; D4 + GNIS and ZCTA; D5 + Overture places (full).

Dataset	Documents	Index	Users within SLO	Broken at	Autocomplete p95	Search p95	Structured p95	Reverse p95
D1	3,280	3 MB	384	not reached	16	21	33	25
D2	783,540	196 MB	384	not reached	16	37	29	31
D3	1,552,532	371 MB	256	512	23	35	50	33
D4	1,573,062	378 MB	256	512	21	28	30	30
D5	1,649,644	435 MB	192	384	26	50	30	23

3.6.1 The same curve for pgeo Pelias's curve alone cannot answer the question a reader actually has, which is whether pgeo's advantage survives more data. So pgeo was rebuilt from each of the same subsets – the definitions are imported from the Pelias harness rather than restated, so the two curves are comparable by construction – and ramped at a fixed 2 vCPU configuration.

Table 36. pgeo capacity against data volume, beside Pelias at the same CPU size for the same subsets. pgeo loses four times its capacity across the range where Pelias loses two.

Dataset	Sources	pgeo features	pgeo users	Pelias users	Pelias / pgeo
D1	whosonfirst	3,289	192	384	2.0x
D2	whosonfirst, openaddresses	680,518	128	384	3.0x
D3	whosonfirst, openaddresses, openstreetmap	808,989	64	256	4.0x

Dataset	Sources	pgeo features	pgeo users	Pelias users	Pelias / pgeo
D4	whosonfirst, openaddresses, openstreetmap, gnis, zcta	829,519	64	256	4.0x
D5	whosonfirst, openaddresses, openstreetmap, gnis, zcta, overture	906,101	48	192	4.0x

Three things follow, and the first is not comfortable for pgeo.

pgeo is about twice as sensitive to data volume as Pelias. Across the full range its capacity falls four-fold, from 192 users to 48, where Pelias falls two-fold, from 384 to 192. That is the expected consequence of the architecture: trigram similarity has to score a candidate set that grows with the number of names in the database, while Elasticsearch answers from an index whose lookup cost barely moves. Anyone extrapolating this study to a much larger region should assume pgeo pays more for the extra data than Pelias does.

But the ratio between them stops widening. It climbs from 4 to four as OpenStreetMap arrives and then holds at four for D3, D4 and D5. The gap is not growing without bound as data is added; it settles, which is a materially different prospect from a trend that keeps getting worse.

Neither engine cares about records it cannot match. D4 adds GNIS and ZCTA – about 20,500 records – and costs pgeo nothing at all, exactly as those sources cost Pelias nothing. What both engines pay for is *matchable names*: OpenStreetMap's streets and venues halve pgeo's capacity and cost Pelias a ramp step, while three quarters of a million address points do comparatively little. A second state should therefore be judged by the names it brings, not by its gigabytes – the same conclusion the Pelias curve reached, now confirmed on both engines.

Caution

pgeo's four-to-one deficit at full Maine data is not a constant: at a tenth of the data it is two-to-one, and it reaches four only once OpenStreetMap's names are loaded. The honest reading is that pgeo's advantage in memory and accuracy is measured at this data volume, and that its capacity will degrade faster than Pelias's as coverage grows. Multi-state deployment remains untested, and this is the result that makes testing it necessary rather than optional.

Adding 780,000 OpenAddresses documents (D2) changed nothing measurable: exact address terms are cheap to match. Adding OpenStreetMap (D3) and Overture places (D5) each cost one ramp step, because venue and place names are the documents that broad text queries match: more of them means more candidates to score per query.

3.7 Query types

Whether either engine pays a latency penalty for difficult input. Accuracy on messy queries is worth little if messy queries are also slow.

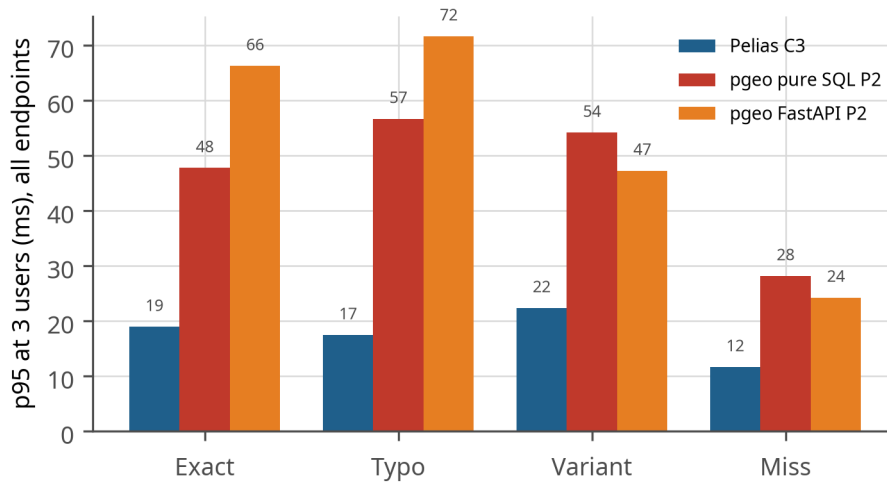


Figure 19. p95 latency at 3 users by query quality (all endpoints together).

Figure 19 shows latency by input quality. Both engines answer misspelled and impossible queries about as fast as exact ones, so the accuracy differences in Section 3.1 are not bought with slower responses.

3.8 Compatibility

Whether an existing Pelias client can be pointed at pgeo without changes. This decides whether pgeo is a replacement or merely an alternative.

	Pelias I	pgeo FastAPI I	pgeo pure SQL I
search basic -	passes	passes	passes
search properties -	passes	passes	passes
search focus -	passes	passes	passes
search boundary.rect -	passes	passes	passes
search boundary.circle -	passes	passes	passes
search circle + focus -	passes	passes	passes
search boundary.gid -	passes	passes	passes
search boundary.country USA -	passes	passes	passes
search boundary.country FRA -	passes	passes	passes
search layers+sources -	passes	passes	passes
search lang + api_key -	passes	passes	passes
search size -	passes	passes	passes
structured -	passes	passes	passes
structured circle -	passes	passes	passes
autocomplete -	passes	passes	passes
autocomplete circle -	passes	passes	passes
autocomplete gid -	passes	passes	passes
reverse -	passes	passes	passes
reverse radius -	passes	passes	passes
reverse properties -	passes	passes	passes
reverse country -	passes	passes	passes
place -	passes	passes	passes
attribution -	passes	passes	passes
error: search without text -	passes	passes	passes
error: bad focus -	differs from contract on Pelias too	passes	passes
error: bad layer -	passes	passes	passes
error: reverse without point -	passes	passes	passes
error: circle without lon -	passes	passes	passes

■ passes
 ■ differs from contract on Pelias too
 ■ fails

Figure 20. Compatibility contract results: each row is a documented Pelias request, each column an engine.

Figure 20 is the compatibility evidence: every documented Pelias request behaves the same on both pgeo front ends. For a reader with existing Pelias clients, this is the row that matters: no client changes are needed.

Case	Request	Pelias	pgeo FastAPI	pgeo pure SQL
------	---------	--------	--------------	---------------

Table 37. Compatibility contract cases.

Case	Request	Pelias	pgeo FastAPI	pgeo pure SQL
search basic	/v1/search?text=389 Congress St, Portland, ME	pass	pass	pass
search properties	/v1/search?text=389 Congress St, Portland, ME	pass	pass	pass
search focus	/v1/search?text=main st&focus.point.lat=44.8016&focus.point.lon=-68.7712	pass	pass	pass
search boundary.rect	/v1/search?text=main st&boundary.rect.min_lat=44.7&boundary.rect.max_lat=44.9&boundary.rect.min_lon=-68.9&boundary.rect.max_lon=-68.6	pass	pass	pass
search boundary.circle	/v1/search?text=main st&boundary.circle.lat=44.8016&boundary.circle.lon=-68.7712&boundary.circle.radius=15	pass	pass	pass
search circle + focus	/v1/search?text=main st&boundary.circle.lat=44.8016&boundary.circle.lon=-68.7712&boundary.circle.radius=15&focus.point.lat=44.8016&focus.point.lon=-68.7712	pass	pass	pass
search boundary.gid	/v1/search?text=congress st&boundary.gid=whosonfirst:locality:85948877	pass	pass	pass
search boundary.country USA	/v1/search?text=bangor&boundary.country=USA	pass	pass	pass
search boundary.country FRA	/v1/search?text=bangor&boundary.country=FRA	pass	pass	pass
search layers+sources	/v1/search?text=portland&layers=locality&sources=wof	pass	pass	pass
search lang + api_key	/v1/search?text=bangor&lang=en&api_key=abc123	pass	pass	pass
search size	/v1/search?text=main st&size=3	pass	pass	pass

Case	Request	Pelias	pgeo FastAPI	pgeo pure SQL
structured	/v1/search/structured?address=389 Congress St&locality=Portland®ion=ME	pass	pass	pass
structured circle	/v1/search/structured?address=Main St&boundary.circle.lat=44.8016&boundary.circle.lon=-68.7712&boundary.circle.radius=15	pass	pass	pass
autocomplete	/v1/autocomplete?text=389 congress	pass	pass	pass
autocomplete circle	/v1/autocomplete?text=main&boundary.circle.lat=44.8016&boundary.circle.lon=-68.7712&boundary.circle.radius=15	pass	pass	pass
autocomplete gid	/v1/autocomplete?text=congress&boundary.gid=whosonfirst:locality:85948877	pass	pass	pass
reverse	/v1/reverse?point.lat=43.6568&point.lon=-70.2626	pass	pass	pass
reverse radius	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&boundary.circle.radius=1	pass	pass	pass
reverse properties	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&layers=address	pass	pass	pass
reverse country	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&boundary.country=US	pass	pass	pass
place	/v1/place?ids=whosonfirst:locality:85948877	pass	pass	pass
attribution	/v1/attribution?	pass	pass	pass
error: search without text	/v1/search?	pass	pass	pass
error: bad focus	/v1/search?text=x&focus.point.lat=999&focus.point.lon=0	differs	pass	pass
error: bad layer	/v1/search?text=x&layers=planet	pass	pass	pass
error: reverse without point	/v1/reverse?	pass	pass	pass
error: circle without lon	/v1/search?text=x&boundary.circle.lat=44.8	pass	pass	pass

Before this work pgeo failed 28 contract checks; the fixes added `boundary.circle`, `boundary.gid`, `boundary.country` and categories filtering, the Pelias hierarchy identifiers (`locality_gid`, `county_gid`, ...), Pelias-shaped error responses from the pure-SQL path, and an edge rule that renames `boundary.circle.lat/lon` (which PostgREST would otherwise read as the focus point's `lat/lon`). Pelias-shaped errors come out of SQL through PostgREST's `response.status` setting; a parameter PostgREST does not know produces a 404, which the edge turns into the Pelias-shaped 400 a client expects.

Remaining differences (docs/PELIAS_COMPATIBILITY.md): OpenAddresses records carry different identifiers in the two engines (Pelias `openaddresses:address:us/me/statewide:<hash>` against pgeo's `openaddresses:address:<hash>`), so a gid saved from Pelias does not resolve in pgeo and clients that store gids must re-resolve them after a switch; categories uses the sources' own category names (Overture's `restaurant`, OpenStreetMap's `natural=water`) rather than the Pelias category taxonomy; and the pure-SQL path rejects parameters it does not know where Pelias ignores them. Pelias also names counties "Cumberland County" where pgeo stored "Cumberland"; pgeo now appends " County" on output so the two agree.

Beyond the geocoding endpoints, Pelias serves two informational HTML pages. `/v1/attribution` lists the data licences, and a client may well fetch it to satisfy the OpenStreetMap and Who's On First terms, so pgeo implements it: the page is built in SQL and served identically by PostgREST and by FastAPI, and it is case 28 of the contract. The `/v1/` page that describes the API itself remains Pelias-only; pgeo answers it with the Pelias-shaped 400 for an unknown endpoint, which is the one place where "an existing Pelias client cannot tell the difference" still does not hold.

3.9 Features in practice

Table 2 in Section 1.4 lists what each platform offers. This section covers the two features that exist only in pgeo and that the LANCER application needs, and what the demo page makes of them.

3.9.1 Structured addresses for LANCER pgeo's `/v1/address` returns a US address in USPS Publication 28[28] form (number, directionals, street name, standard suffix, unit, city, state, ZIP; the delivery and last lines) with the municipality, county and FIPS codes, for a selected search result, a typed address or a map position. When the selected result is not an address (a business or landmark, for example "Just in Time, Lewiston Maine"), it returns the nearest address point with its distance, as the project's requirement asked. The street parser gives a standard suffix for 97.1% of Maine's 41,868 street names; the rest have none in Publication 28 terms (Broadway, Rue Principale). Units come only from the caller, because search merges a building's per-unit records. Queries carry client addresses, so PostgreSQL never logs query parameters.

3.9.2 ZIP+4 The USPS ZIP+4 file[29] would add ZIP+4 codes, USPS street spellings and the USPS preferred city per ZIP (for example SOUTH PARIS for 04281, where the open data says PARIS). It costs \$120 a year for one state (\$1,750 for all), and its licence allows "internal corporate or personal use on one computer at one location", with no network distribution without a paid amendment (unlimited: \$24,000) and no use of data older than 105 days. The project decided against it; the address API stays on open data and is not a deliverability check (docs/ADDRESS_API.md).

What this means

For LANCER this endpoint replaces manual address tidying: a dispatcher picks a business by name and the system stores a correctly formatted mailing address, the town, the county and its FIPS code, with the nearest street address when the place itself has none.

The demo page exposes these features: autocomplete with map-centre bias, structured search, reverse geocoding by map click, a CSV batch tool, and, where both engines are deployed, an engine switch, an Address tab (type-ahead with a best-match suggestion, the USPS block with a copy button, and the nearest

street address drawn on the map for venues) and a Compare tab that runs one query on both engines side by side.

3.10 Parity between the platforms

Do the two platforms now offer the same product? Feature for feature through the documented API, yes; in accuracy, pgeo is ahead rather than level; in throughput, Pelias is ahead.

Table 38. Feature parity through the documented Pelias API.

Area	Both	Only pgeo	Only Pelias
Endpoints	search, structured, autocomplete, reverse, place, attribution	/v1/address (USPS Publication 28, nearest street address)	/v1/nearby (beta), the /v1/description page
Parameters	text, size, layers, sources, focus, boundary.rect, boundary.circle, boundary.country, boundary.gid, categories, lang and api_key accepted		multilingual names (lang changes the output)
Response	GeoJSON envelope, Pelias properties, hierarchy identifiers (*_gid, county_a), errors as HTTP 400 with geocoding.errors	confidence lowered for ties between distinct places (better calibrated)	
Parsing	US addresses	rule parser tuned for Maine input	libpostal: international address parsing
Categories	the categories filter		the Pelias category taxonomy (food, health, ...)
Scale	Maine (tested)		planet-scale data
Differences to handle		pure-SQL front end rejects unknown parameters (400)	OpenAddresses records have different identifiers than in pgeo

Table 39. Accuracy parity by category: Pelias against pgeo (pure SQL), all query qualities. Pelias leads only where both are near perfect.

Category	Pelias	pgeo	Gap (points)	Ahead
Addresses	88%	98%	+10	pgeo
Towns	81%	96%	+15	pgeo
Lakes, summits	37%	87%	+50	pgeo
Venues	60%	94%	+34	pgeo
ZIP codes	100%	98%	-2	Pelias
Reverse	100.0%	99.5%	-0.5	Pelias
Misses	41%	95%	+53	pgeo
All cases	75.5%	95.8%	+20.3	pgeo

Table 40. Throughput parity: most concurrent users within all latency targets at each CPU size, with each configuration’s memory budget.

CPU	Pelias users (budget)	pgeo pure SQL	pgeo FastAPI	Pelias / best pgeo
1 vCPU	96 (8.3 GB)	24 (1.6 GB)	24 (1.6 GB)	4.0x
2 vCPU	192 (8.4 GB)	48 (2.5 GB)	48 (2.7 GB)	4.0x
4 vCPU	384 (10.8 GB)	96 (3.0 GB)	96 (3.3 GB)	4.0x

In short: an existing Pelias client can switch to pgeo without code changes (the contract in Section 3.8) and gets more correct answers, especially for misspelled and vague queries, at a fraction of the memory; it gives up throughput per CPU, multilingual names, international parsing and planet scale, none of which this deployment uses.

3.11 Production validation on VM 120

Every number so far comes from the workstation, where the engine and the load generator are pinned to separate physical cores and nothing else competes. That isolation is what makes the comparison fair, and it is also what makes it artificial: the service people will actually use runs on VM 120 behind the wharf Caddy, so the last measurement repeats the three-user validation and the ramp against the deployed service over HTTPS, one engine at a time, with the edge’s rate limits raised for the window and restored afterwards.

Table 41. Both engines on VM 120 through the production HTTPS path: the three-user validation, the p95 per endpoint and how far the ramp went.

Engine	3 users	Autocomplete p95	Search p95	Structured p95	Reverse p95	Users within targets	Broken at
pelias	pass	19	22	24	10	384	not reached
pgeo	pass	32	60	62	16	128	256

Two differences from the controlled runs are built in and worth keeping in mind when comparing the numbers. The request now crosses TLS termination, the nginx edge and the LAN, which adds a few milliseconds that belong to the service rather than to the engine; and the VM’s cores are two CPU generations newer than the workstation’s (Section 1.2), which works the other way. The purpose here is not a cleaner comparison between engines but a check that the service, as deployed, meets the targets it was designed for.

Both engines pass the three-user validation with room to spare: the worst endpoint reaches 8% of its target on Pelias and 13% on pgeo, with no errors. For the service this project set out to build, that is the answer – either engine, on the hardware already owned, is far inside its requirement.

The ramp is more interesting than the validation, because it disagrees with the workstation in a useful way. Pelias holds 384 concurrent users here and pgeo 128, a ratio of three rather than the four measured under controlled conditions (Table 40). Both engines do better on the VM than on the workstation, which is expected – VM 120’s cores are two CPU generations newer and the engine does not share the machine with the load generator – but pgeo gains more from the newer core than Pelias does. That is consistent with where each spends its time: pgeo runs interpreted PL/pgSQL, which a faster core speeds up directly, while Pelias’s compiled index lookups were already cheap per request.

What this means

Take the ratio, not the absolute numbers, from this report. The four-to-one throughput gap measured under controlled conditions narrows to three-to-one on newer cores, and both engines serve far more users on the production VM than on the test workstation. Sizing from the tables here will therefore be conservative on comparable hardware, which is the direction an operator wants to be wrong in.

3.12 The smallest server that works

The headline question of this project is not which engine is fastest but how small a machine the three-user service can run on. A search program answered it directly (`tests/load/find_floor.py`): start from a configuration known to pass, shrink one resource at a time – first the CPU quota, then each service's memory – and keep a step only when a five-minute three-user run stays within every latency target with no errors, no container restarts and no out-of-memory kill. A step that fails ends the search for that resource, because smaller values of it will fail too. Table 42 is the whole path, pass and fail.

Table 42. Minimum-server search: every trial, in order, for both engines. Memory totals include 0.8 GB for the operating system.

Engine	Trial	Configuration (vCPU quota; memory limits in GB)	Total	3 users	Result
pgeo (pure SQL)	0	cpus 1, db 0.6, rest 0.25	1.65 GB	pass	ok
pgeo (pure SQL)	1	cpus 0.5, db 0.6, rest 0.25	1.65 GB	pass	ok
pgeo (pure SQL)	2	cpus 0.25, db 0.6, rest 0.25	1.65 GB	pass	ok
pgeo (pure SQL)	3	cpus 0.25, db 0.45, rest 0.25	1.50 GB	pass	ok
pgeo (pure SQL)	4	cpus 0.25, db 0.35, rest 0.25	1.40 GB	fail	over target (worst 1.07x, errors 0)
pgeo (pure SQL)	5	cpus 0.25, db 0.45, rest 0.15	1.40 GB	pass	ok
pgeo (pure SQL)	6	cpus 0.25, db 0.45, rest 0.1	1.35 GB	fail	over target (worst 1.13x, errors 0)
Pelias	0	cpus 1, es 1.5, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.8	8.32 GB	pass	ok
Pelias	1	cpus 0.5, es 1.5, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.8	8.32 GB	pass	ok
Pelias	2	cpus 0.25, es 1.5, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.8	8.32 GB	pass	ok

Engine	Trial	Configuration (vCPU quota; memory limits in GB)	Total	3 users	Result
Pelias	3	cpus 0.25, es 1.2, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.8	8.02 GB	pass	ok
Pelias	4	cpus 0.25, es 1, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.8	7.82 GB	pass	ok
Pelias	5	cpus 0.25, es 0.8, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.8	7.62 GB	fail	died or OOM
Pelias	6	cpus 0.25, es 1, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.6	7.62 GB	pass	ok
Pelias	7	cpus 0.25, es 1, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.45	7.47 GB	pass	ok
Pelias	8	cpus 0.25, es 1, api 0.42, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.35	7.37 GB	pass	ok
Pelias	9	cpus 0.25, es 1, api 0.42, libpostal 2, interpolation 2.1, pip 0.55, placeholder 0.35	7.22 GB	fail	died or OOM
Pelias	10	cpus 0.25, es 1, api 0.42, libpostal 2, interpolation 1.8, pip 0.7, placeholder 0.35	7.07 GB	fail	died or OOM
Pelias	11	cpus 0.25, es 1, api 0.42, libpostal 1.8, interpolation 2.1, pip 0.7, placeholder 0.35	0.00 GB	fail	did not start: RuntimeError
Pelias	12	cpus 0.25, es 1, api 0.3, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.35	7.25 GB	pass	ok

Engine	Trial	Configuration (vCPU quota; memory limits in GB)	Total	3 users	Result
Pelias	13	cpus 0.25, es 1, api 0.22, libpostal 2, interpolation 2.1, pip 0.7, placeholder 0.35	7.17 GB	pass	ok

pgeo ran the three-user load inside **0.25 vCPU and 1.40 GB** of memory in total: a quarter of one core, 0.45 GB for PostgreSQL, 0.15 GB for PostgREST and 0.8 GB left for the operating system. Every failed step failed on latency, not on memory: the database kept answering at 0.35 GB and at 0.1 GB for the front end, just too slowly (7 to 13 percent over target). That is the practical signature of a database whose working set is smaller than the machine: it degrades gently instead of being killed.

Pelias floors at **0.25 vCPU and 7.17 GB**. Its limit is the opposite kind: the memory is not a tuning choice but the sum of what five services load before the first request (the libpostal model, the interpolation database, placeholder's in-memory tables and the Elasticsearch heap), so the floor barely moves however little traffic it serves.

3.12.1 Two kinds of limit The trials fail in two distinct ways, and the difference explains why the two engines' floors are so far apart. pgeo's failures are all of the first kind: the service keeps answering, just too slowly, so its floor is wherever the operator decides the latency is no longer acceptable. Most of Pelias's are of the second: below a threshold the service does not run at all. libpostal cannot even start below 2 GB, because it loads a fixed model before it serves anything; the interpolation service crash-loops at 1.8 GB; Elasticsearch is killed at 0.8 GB. Those are properties of the software, not of the load, and no amount of tuning for three users moves them.

3.12.2 A floor is a pair, not a number The two resources are not independent, and the search makes that visible: Pelias's pip service was killed at 0.55 GB in this search, although the capacity tests had it surviving at 0.4 GB. Nothing regressed – in the capacity tests each service had a whole core, and here it has a quarter of one, so collection and compaction fall behind and the same work needs more headroom. Read each floor as the pair of numbers it is: at more CPU both engines would fit in less memory, and at less memory they would need more CPU to stay inside the targets.

Key result

The three-user service fits in 1.40 GB on pgeo and 7.17 GB on Pelias – a factor of five. Both engines idle comfortably below a quarter of a core, so at this scale the machine is chosen by memory, not by CPU – which is why the two platforms land in different price classes (Table 34).

3.12.3 Confirmation on real machines A container limit is not a server: it shares the host's page cache, its disk and its kernel. Each floor was therefore re-tested on a temporary Proxmox VM of that size, created for the run and destroyed after it (tests/load/floor_vm.sh), with the same three-user validation followed by a ramp to find how far that VM stretches.

Table 43. The measured floor confirmed on a temporary VM: the three-user validation and the ramp to the first configuration outside a latency target.

Engine	VM CPU	VM memory	3 users	Worst p95 / target at 3	Users within targets	Broken at
pgeo (pure SQL)	1 vCPU (limit 1)	1 GB	pass	0.21	32	96

The container search said pgeo needs 0.60 GB; on a real 1 GB machine it used 307 MB, and the three-user load ran with the worst endpoint at 21% of its target. A cheap VM is not a container limit – it brings its own kernel, page cache and disk – and pgeo came out ahead of the container estimate rather than behind it, because a minimal operating system leaves more for the database than the 0.8 GB this study reserves. The same machine held 32 concurrent users before an endpoint missed its target.

Key result

pgeo runs the Maine geocoder on the smallest virtual machine providers sell: 1 vCPU and 1 GB, about \$5 a month, with the three-user workload at a fifth of its latency budget and room for 32 concurrent users. Pelias needs 7.17 GB, which is an 8 GB plan at roughly ten times the price.

Pelias's floor was not re-tested on a temporary VM: at 7.17 GB it needs a guest the Proxmox host could not spare beside the machines already running on it. Its real-machine evidence is VM 120 itself (Section 3.11), which runs Pelias in 10 GB and serves 384 users; the floor search is what says how far that could be cut.

3.13 Security

The two platforms differ in security the way they differ in everything else: Pelias is more software, and pgeo is more of this project's own software. Both statements cut. The controls below were measured on the deployed service rather than read off the configuration, and `tests/security/test_posture.py` asserts each one on every build and every deploy.

Table 44. Security properties of each platform as deployed. Measured 2026-09-20.

Property	Pelias	pgeo
Processes reachable over HTTP	6 (API, Elasticsearch, libpostal, placeholder, pip, interpolation)	2 (PostgREST, PostgreSQL)
Data-store credentials	none: Elasticsearch accepts unauthenticated reads and writes from anything that reaches the port	password, pg_hba, and an API role holding SELECT only
Effect of a foothold on the host	the whole index: read, modify or delete	whatever SELECT allows; the API role has no write path
Attack surface in this project's code	none; the query path is upstream	the ranking SQL, the edge parameter rewriting and /v1/address
Components to patch	6 pinned images	2 pinned images
Query text in engine logs	not logged	not logged (<code>log_parameter_max_length = 0</code> , <code>log_statement = none</code>)

3.13.1 What the two columns are really saying Pelias's data store has no credentials at all: its design assumes nothing can reach it, so the loopback binding is the entire access-control story. That is a property of its architecture, not a defect in its code, and it is why the single cheapest improvement available to a Pelias deployment is enabling Elasticsearch's own authentication – configuration, not another container.

pgeo authenticates its data store and gives the HTTP layer a role that cannot write, so the same foothold yields less. Set against that, pgeo's query path is code written for this project: its ranking SQL and its edge rules have one set of eyes on them, where Pelias's have many.

Shared posture. Both engines sit behind the same edge, and that is where most of the protection lives: every engine port bound to loopback, a firewall that denies inbound by default and admits the TLS terminator alone, GET only, per-client rate limits, a content security policy naming no remote origin, and a demo page that builds every element with `textContent` – it contains no `innerHTML`, `eval` or `document.write` at all, which is what makes the query echoed in each response harmless.

Key result

When this study was first written up, neither engine authorized anything: any device that could reach the edge could resolve any address, which for the LANCER use case meant any device on the LAN could resolve a client address. That was the one place where “fewest components” and good security genuinely conflicted, and the lightest honest answer – hashed API keys checked at the edge, issued out of band – was proposed as costing no new component. It is now done (Section 3.13.2), and it cost none.

3.13.2 Authorization at the edge Every `/v1/*` request must carry a key in an `X-API-Key` header. The edge hashes the header with SHA-256 – an `nginx` module, `njs`, not a new process – and looks the hash up in a map of known clients; no match, no API. The map is rendered from the deployment inventory, which holds hashes and client names and nothing else; the keys themselves are shown once by the tool that makes them (`scripts/edge_apikey.sh`) and kept by the client. A key is 32 bytes from the operating system's random source, which is why a plain SHA-256 is the right store for it: a slow password hash defends low-entropy secrets people choose, and there is nothing low-entropy here.

Four decisions in it are worth stating, because each closes a leak that a simpler design leaves open:

- **Header, not URL.** Pelias clients often send `api_key=` in the query string. A URL lands in browser history, in referers and in any proxy's log; the edge refuses a key there – a valid one included – rather than accepting it.
- **The log names the client, never the key.** The access-log format that already omitted the query string now records the client's name where the combined format has the user, so an audit trail exists (“dispatch made 4,000 requests on Tuesday”) and a stolen log yields no credential.
- **Fails closed.** With the check on and no keys configured, the deployment refuses to proceed rather than serve everyone. The off switch exists for a LAN demo, is named in the rendered configuration, and the security tests fail against any host that has it on.
- **The demo page is a client too.** It asks for a key, keeps it for the tab only (never `localStorage`, never a cookie), and sends it in the header like any other client. Anything else would have been a hole shaped exactly like the threat.

Static files, tiles and the page shell need no key: they hold no data. The two rate limits stay, keyed by client address, and bound how fast anyone can guess – not that guessing a 256-bit key is a plan. 37 tests hold the behaviour, against a live edge (refusals with and without a key, malformed keys, the query-string case, every endpoint with a good key, the log) and against the configuration that produces it (every proxied location guarded before it proxies; hashes only; fail closed; the switch named).

What it does not do: identify people. A key names a *client* – the dispatch application, the demo – not a user of it. Single sign-on for the people behind those clients remains future work (Section 6), and it is a different problem: this one was about which machines may ask; that one is about which people.

A finding worth repeating, because of how it hid. PostgreSQL was deliberately configured not to log bind parameters, since a query here is an address someone searched for and for LANCER a client address.

That control was real. The nginx access log in front of it, however, used the default format, which records the whole request line – so every address searched was written to disk in plaintext anyway. A control at one layer was undone by a default at the next, and the system-level claim looked satisfied the whole time. The edge now logs the path without its query string, verified on the deployed service.

Why the deployment shape carries so much of the weight. Both engines serve public open data, so a total compromise of the index or the database leaks nothing that is not already downloadable. The sensitive material is in the queries, not the corpus, which is why a logging default mattered more here than anything about the data store. Self-hosting on a LAN removes rather than mitigates whole classes of threat – no bot traffic, no scraping, no credential stuffing, no exposure to internet-wide scanning – and that is what makes a two-container service on a 1 GB machine a defensible posture rather than a reckless one. The cost is that patching becomes the department’s job: six images or two, and nobody updates them in the background. Deployment scenarios at three levels of ambition, and where each control pulls against the fewest-components principle, are in docs/SECURITY_DEPLOYMENT.md.

3.14 Photon, the type-ahead specialist

Section 1.5.1 named the comparison this study was most exposed to: pgeo is limited by autocomplete on every configuration, and Photon[3] is the engine built specifically to do autocomplete quickly. This section measures it, on the same machine, with the same corpus, the same session mix, the same keystroke pattern, the same think times, the same ramp and the same latency targets as the other two engines. Only the URL shape differs, because Photon does not speak the Pelias API (tests/load/k6/session_photon.js).

The result is not the one the caveat anticipated. Photon is faster per request than pgeo at type-ahead, as expected – but it is *less* CPU-efficient than Pelias, so it does not become the harsher yardstick for capacity. What it does expose is a data limitation that matters more than either.

3.14.1 How Photon was built A Photon index is not built from source data directly: it is exported from a Nominatim database[4]. The Maine OpenStreetMap extract was imported into Nominatim 4.5 (about eight minutes), and Photon’s importer then produced its own index in **39 seconds – 791,593 documents, 203 MB on disk**. That is by far the fastest index build in this report, and the comparison is not quite like for like: the eight-minute Nominatim import is a prerequisite, and for a planet deployment Photon distributes a prebuilt index that an operator downloads rather than builds at all.

The import path is also the constraint. Nominatim imports OpenStreetMap, so a Photon index contains OpenStreetMap and nothing else. The OpenAddresses E911 address points (780,260 documents in Pelias) and the Overture places (76,582) that both other engines carry cannot be in it. Table 48 shows what that costs.

3.14.2 Capacity **Table 45. Concurrent users each engine holds inside the latency targets, on two cores and unconstrained.** Photon sits between the other two on capacity and below both on memory.

Configuration	vCPU	Users within targets	Broke at	Throughput at the limit	Memory under load
Pelias C2	2	192	384	209 req/s	8.4 GB budget
Photon (pinned)	2	128	256	127 req/s	1.84 GB resident
pgeo rest-P2	2	48	96	53 req/s	2.5 GB budget
Pelias M0	all 12	192	384	215 req/s	-
Photon (unpinned)	all 12	384	512	404 req/s	1.84 GB resident
pgeo rest-PM	all 12	128	256	144 req/s	-

On two cores Photon holds 128 concurrent users to Pelias's 192 and pgeo's 48. Two things follow. Photon carries **2.7 times** as many users as pgeo on the same CPU – a real gap, and a smaller one than the four times pgeo gives up to Pelias. And Photon is **not** the CPU-efficiency leader: Pelias holds half again as many users on the same two cores. Unconstrained, Photon reaches 384 users, which is what Pelias reaches on four cores; Pelias's own unconstrained figure is held down to 192 by its single API worker rather than by Elasticsearch.

The memory column is where Photon is unambiguously strong: 1.84 GB resident under full load, against the 8.4 GB budget Pelias needs for its 192 users. Per gigabyte, Photon is the most efficient engine measured here.

3.14.3 Latency at type-ahead Table 46. Autocomplete p95 on two cores. The 250 ms target is the binding one for pgeo. Two figures are given for pgeo where two runs were recorded.

Concurrent users	Pelias C2	Photon (2 cores)	pgeo rest-P2
48	21 ms	53 ms	103 / 189 ms
96	36 ms	92 ms	1,211 / 1,453 ms
128	43 ms	228 ms	past its limit
192	174 ms	770 ms	past its limit

At 48 users – pgeo's ceiling – Photon answers a keystroke in 53 ms against pgeo's 103 to 189 ms and Pelias's 21 ms. So the specialist is about two to three times quicker per keystroke than pgeo, and about two and a half times slower than Pelias's dedicated autocomplete endpoint. The caveat in Section 1.5.1 was right that pgeo is the slowest of the three at type-ahead and wrong that Photon would prove the harshest comparison.

3.14.4 Accuracy, and why the speed is not the whole story The same 1,560 accuracy cases were run against Photon, scored identically – same ground truth, same 60 to 150 m radius, same housenumber check (tests/accuracy/run_accuracy_photon.py).

Table 47. Accuracy on the Maine test set. Photon's 58% at autocomplete is the endpoint it is built for, which is the clue that the cause is data rather than engine.

Endpoint	Cases	Pelias	pgeo	Photon	Photon returns nothing
Search	1,060	66.3%	94.6%	71.3%	19.7%
Autocomplete	150	86.7%	95.3%	58.0%	9.3%
Structured	150	96.7%	100.0%	88.7%	3.3%
Reverse	200	100.0%	99.5%	87.0%	0.5%
All	1,560	75.5%	95.8%	73.7%	14.7%

Photon scores 58% on autocomplete – the endpoint it exists to serve – and that number is the signal that something other than query quality is being measured. Breaking the same results down by what was being searched for makes the cause plain:

Table 48. Accuracy by feature kind. Photon collapses on venues because Overture places cannot enter an OpenStreetMap-only index, not because it ranks them badly.

What was searched for	Cases	Source of the ground truth	Pelias	pgeo	Photon
Venues	225	Overture Maps places	60.4%	94.2%	19.1%
Lakes and summits	150	USGS GNIS	36.7%	86.7%	74.0%
Addresses	550	OpenAddresses E911	87.8%	97.8%	83.6%
Reverse addresses	200	OpenAddresses E911	100.0%	99.5%	87.0%
Towns	225	Who's On First	80.9%	95.6%	89.3%
ZIP codes	60	Census ZCTA	100.0%	98.3%	95.0%
Deliberate misses	150	none – should not match	41.3%	94.7%	69.3%

Venues are the collapse: 19.1%, against 94.2% for pgeo. The venue ground truth is Overture Maps places, and Photon's index is built from Nominatim, which imports OpenStreetMap only – those businesses and landmarks are simply not in it to be found. The same mechanism, more weakly, explains the address and GNIS gaps. This is a statement about what a Photon index *can contain* for Maine, not about how well Photon searches what it holds.

The deliberate misses are a different limitation and an architectural one. A miss counts as correct when an engine returns nothing or scores its best guess below 0.8 confidence. Photon returns no confidence score at all, so it can only ever satisfy the first condition: it has no way to tell a client “here is a result, but I do not believe it”. For LANCER, where a wrong address dispatched confidently is the failure that matters (Section 3.9.1), that is a disqualifying property independent of speed.

Caution

Photon is quick and remarkably light on memory, and on OpenStreetMap data it is a capable geocoder. It is not a candidate for this project: its index cannot contain the E911 address points or the Overture places that Maine dispatch depends on, and it cannot express uncertainty. The measurement resolves the open question from Section 1.5.1 – pgeo's type-ahead is two to three times slower per request than the specialist's – without changing the recommendation.

3.15 Nominatim, the reference implementation

Nominatim[4] is the OpenStreetMap geocoder – the one that answers openstreetmap.org, and the database a Photon index is exported from. It is also the closest relative of pgeo in the survey of Section 1.6: it computes addresses inside PostgreSQL with PL/pgSQL triggers and searches from a Python application, where pgeo's search *is* the database. Having built it to make the Photon index, measuring it cost one afternoon and closes the OpenStreetMap side of the comparison.

It was measured on two cores, pinned the same way as Pelias C2 and pgeo P2, with the same corpus, session mix, keystroke pattern, think times, ramp, latency targets and accuracy cases. Only the URL shape differs (tests/load/k6/session_nominatim.js). Nominatim 4.5 served the requests from four Unicorn workers in front of its own PostgreSQL, so unlike Pelias's unconstrained configuration its front end was not the bottleneck.

3.15.1 Capacity **Table 49. Concurrent users each engine holds inside the latency targets, on the same two cores.** Nominatim lands between Photon and pgeo, and closer to pgeo.

Engine, two cores	Users within targets	Broke at	Throughput at the limit	Memory under load
Pelias C2	192	384	209 req/s	8.4 GB budget
Photon	128	256	127 req/s	1.84 GB resident
Nominatim	64	128	68 req/s	2.19 GB resident
pgeo rest-P2	48	96	53 req/s	2.5 GB budget

Nominatim holds 64 concurrent users to pgeo's 48. That is the narrowest gap in this report: **pgeo is within 1.33 times of the reference OpenStreetMap geocoder**, against four times for Pelias and 2.7 times for Photon. (These are the figures as measured at the time. The optimization reported in Section 3.16 later took pgeo to 64 users on the same two cores - level with Nominatim.) Throughput saturates at about 75 req/s – it was 68 req/s at 64 users, 75 at 96 and 77 at 128, while latency went from 0.62 to 3.05 to 5.26 times target, which is the signature of a queue rather than of a cliff.

The comparison flatters neither engine unfairly. Nominatim and pgeo are the two architectures in this study that put the work in PostgreSQL, and they land within a third of each other on the same hardware. The difference is where the *search* runs: Nominatim's is a Python application issuing queries, pgeo's is the query.

3.15.2 Latency **Table 50. Autocomplete p95 on two cores, against the 250 ms target.** Nominatim has no autocomplete endpoint, so these are full searches on a growing prefix.

Concurrent users	Pelias C2	Photon	Nominatim	pgeo rest-P2
48	21 ms	53 ms	85 ms	103 / 189 ms
64	-	-	155 ms	past its limit
96	36 ms	92 ms	762 ms	past its limit
128	43 ms	228 ms	1,315 ms	past its limit

At 48 users Nominatim answers a keystroke in 85 ms against pgeo's 103 to 189 ms – quicker, but the same order, and both far behind Pelias's 21 ms. What the table cannot show is that Nominatim's answers to those keystrokes are mostly empty, which is the subject of the next section.

3.15.3 Accuracy, and a failure mode worth naming The same 1,560 cases, the same ground truth, the same scoring (tests/accuracy/run_accuracy_nominatim.py).

Endpoint	Cases	Pelias	pgeo	Photon	Nominatim	Nominatim returns nothing
----------	-------	--------	------	--------	-----------	---------------------------

Table 51. Accuracy on the Maine test set. The right-hand column is the important one: Nominatim's errors are overwhelmingly empty answers rather than wrong ones.

Endpoint	Cases	Pelias	pgeo	Photon	Nominatim	Nominatim returns nothing
Search	1,060	66.3%	94.6%	71.3%	61.3%	45.9%
Autocomplete	150	86.7%	95.3%	58.0%	8.0%	75.3%
Structured	150	96.7%	100.0%	88.7%	86.0%	4.7%
Reverse	200	100.0%	99.5%	87.0%	86.0%	0.0%
All	1,560	75.5%	95.8%	73.7%	61.7%	38.9%

8.0% at autocomplete, with three-quarters of those queries returning nothing at all. This is not a defect; it is the absence of a feature, measured. Nominatim has no autocomplete endpoint, so a keystroke prefix goes to /search, and /search on two or three characters matches nothing. This is the clearest evidence in the report for why Photon exists: Photon is the OpenStreetMap world's answer to precisely this gap, and Section 3.14 shows it scoring 58% where Nominatim scores 8%.

The right-hand column deserves attention beyond autocomplete, because it describes Nominatim's character as a geocoder:

Table 52. Accuracy by feature kind. Nominatim's median error on addresses is zero metres: when it answers, it is exact. Its failures are almost entirely recall.

What was searched for	Cases	pgeo	Nominatim	Nominatim finds nothing	Median error when it answers
Addresses	550	97.8%	69.1%	22.0%	0 m
Reverse addresses	200	99.5%	86.0%	0.0%	0 m
Towns	225	95.6%	49.3%	38.2%	96 m
Lakes and summits	150	86.7%	54.0%	44.7%	36 m
Venues	225	94.2%	8.9%	84.0%	163 m
ZIP codes	60	98.3%	95.0%	3.3%	2,627 m
Deliberate misses	150	94.7%	94.7%	94.7%	-

Two readings follow, and they point the same way.

Nominatim is precise and does not reach. Where it answers an address, the median error is **zero metres** – it returns the same point the ground truth came from. The town radius is 8 km and its median town error is 96 m, so the towns it misses are not near-misses: it simply finds nothing for 38% of them. Names like “richs corner” and “holeb” exist in Who's On First and GNIS but are not searchable places in OpenStreetMap. Venues collapse to 8.9% for the reason Section 3.14 gives for Photon: the venue ground truth is Overture Maps places, and an OpenStreetMap-only index cannot contain them.

Its best score is an artefact of its worst habit. Nominatim scores 94.7% on the deliberate misses – equal to pgeo, the highest in the study. But it earns that by returning nothing for 94.7% of them, and it returns nothing for 38.9% of *all* queries including the ones that had a right answer. pgeo reaches the same 94.7% while returning nothing for only 39.3% of misses, because it can answer with a low confidence score instead of silence. The same conservatism that makes Nominatim look disciplined on misses is what costs it 39

points of recall elsewhere.

Key result

Nominatim is the fairest comparison in this report for pgeo's architecture – both put the work in PostgreSQL – and pgeo comes within 1.33 times of it on capacity while scoring 95.8% against 61.7% on this test set. That accuracy gap is mostly data rather than engine: Nominatim indexes OpenStreetMap alone, where pgeo carries OpenAddresses, Overture, GNIS, Who's On First and ZCTA. The part that is not data is the missing autocomplete endpoint, and that is a real architectural difference rather than a matter of loading more sources.

3.15.4 What the four engines together say **Table 53. The four engines measured in this study, on the same hardware, data and test set.** Sources and endpoints explain more of the accuracy column than engine quality does.

	Pelias	Photon	Nominatim	pgeo
Users on two cores	192	128	64	48
Memory under load	8.4 GB	1.84 GB	2.19 GB	2.5 GB
Accuracy, 1,560 cases	75.5%	73.7%	61.7%	95.8%
Autocomplete accuracy	86.7%	58.0%	8.0%	95.3%
Data sources	6	OSM only	OSM only	6
Dedicated autocomplete	yes	yes	no	yes
Expresses uncertainty	yes	no	no	yes
Processes to run	6	1	2	2

Read down the accuracy row and the temptation is to conclude pgeo is simply the better geocoder. Read the two rows beneath it and the honest conclusion is narrower: pgeo and Pelias carry six sources where Photon and Nominatim carry one, and for Maine that one source is missing the E911 address points and the Overture places the test set is built from. What pgeo can claim against the OpenStreetMap engines is not that it searches better, but that it can hold data they cannot.

What survives that caveat is the capacity result, which is measured on the same machine regardless of what each engine holds: **pgeo costs four times Pelias per CPU, 2.7 times Photon, and 1.33 times Nominatim** - as first tuned. Section 3.16 then removed a third of pgeo's autocomplete cost without changing an answer, which makes those three times, two times, and parity: against the engine closest to its own architecture, the measured price of putting the whole query path in SQL is now nothing the ramp can resolve.

3.16 Optimization: a third more capacity, with no answer changed

Every capacity figure so far describes pgeo as it was first tuned - by configuration and one missing index (Section 3.2). The SQL of the query path itself had never been profiled. This section reports what profiling it found, and how the changes were proven not to alter a single answer, which for this project is the part that matters: accuracy is the reason to choose pgeo, so a faster engine that answers differently would be a worse one. The working log, with every measurement, is `docs/PERFORMANCE_OPTIMIZATION.md`.

3.16.1 Where the time went Autocomplete sets pgeo's capacity (Section 3.4.3) and pgeo is CPU-bound (Section 3.4.4), so the target was CPU per autocomplete request. It was measured over a seeded workload of 2,920 keystrokes - 140 type-ahead sessions with the load test's own mix, every prefix of every text.

Table 54. The four changes made to the autocomplete query path. None alters what the query means; each removes work whose result was discarded or never needed.

Finding	What was happening	The change	Effect on the stage
F1. A filter that filtered nothing	<code>keep()</code> applies the optional layer, source and boundary filters. It holds an <code>EXISTS</code> , so PostgreSQL cannot inline it; it ran once per candidate, each call materialising the whole 780-byte row. With no filter set - the normal request - it returned true every time	Decide once per request whether any filter is set, and skip the call when none is	127 ms -> 48 ms for a common prefix
F2. Ten results kept, 376 built	A 27-field result record was assembled for every candidate, and the spheroidal distance to the focus computed twice per row; then all but size were discarded	Rank on a plain expression with one distance; build records for the rows that survive the <code>LIMIT</code>	42 ms of ranking for rows that were thrown away
F3. Hot columns behind cold ones	Ranking reads five columns that sit behind some twenty variable-length ones in a 780-byte row, and PostgreSQL walks every preceding variable-length field to reach them	A narrow side table for autocomplete, hot columns first: 188 bytes a row, 60 MB, built in 1.2 s. The same idea the engine already used for streets	7,809 heap pages -> 2,912; 48 ms -> 14 ms
F4. One letter, every street	While a word is typed its prefix is searched as <code>s:*</code> . GIN expands that to every token starting with “s” - including “street”, in nearly every address - and unions them all before the selective words narrow anything	When the word being typed is one or two letters, ask the index for the complete words only and test the prefix on the rows that come back	44.8 ms -> 2.9 ms for the same 54 rows

Together they cut autocomplete CPU by **36%** (50.3 s to 32.2 s over the workload). search received F1 and the single distance computation; it is not the binding endpoint and has fivefold headroom on its target.

3.16.2 How “no answer changed” was established An unchanged accuracy percentage is too weak a test. It cannot see one case breaking while another is fixed, nor an answer changing to a different correct one. The claim tested was the stronger one - **nothing a client can observe changed** - and the evidence was gathered before any code was touched.

The complete features JSON, every field of every result, was recorded for **7,380 queries**: the 1,560 accuracy cases and 1,800 fuzz cases on all four endpoints; the 2,920 keystrokes; and 1,100 filtered and focused variants (layers, sources, rectangle, circle, gid, sizes 1 to 40), because the changes add a fast route for unfiltered requests and the slow route needed its own proof. The original was first run against itself - 7,380 of 7,380 identical - so it is deterministic and any later difference is attributable to the change.

Table 55. Verification of the optimized engine. “Case by case” compares verdict, hit@1, hit@5, distance to the truth, confidence, error and result count for each case, rather than the totals.

Check	Scope	Result
Golden set, full JSON, direct SQL	7,380 queries, 47,962 result rows	byte-identical
Accuracy harness over HTTP, pure SQL, case by case	3,360 cases	0 changed
Accuracy harness over HTTP, FastAPI, case by case	3,360 cases	0 changed
Accuracy / fuzz score	both front ends	95.8% / 66.7%, before and after
Accuracy gate, Pelias compatibility contract, security posture		pass
Full rebuild from raw data, then the gate	committed build script	gate passed, 95.8% / 66.7%; verdict unchanged on all 3,360 cases

3.16.3 What the verification caught Three things, each of which the accuracy percentage alone would have let through. They are reported because they are the argument for the method.

A build defect. The side table was first filled with `INSERT ... ORDER BY ctid`. PostgreSQL’s insert backfills earlier pages with rows that fit, and **660 of 203,399 rows landed out of physical order**. That matters because of a property of the engine this work exposed: *where candidates tie - two sources for one pond with the same label and importance - the winner is decided by the order rows reach the sort, which is physical order*. Twelve of 4,018 autocomplete results changed, every one a tie falling the other way. `CREATE TABLE AS` bulk-appends and leaves none out of order; the twelve went to zero, and a test now guards the order.

A change that did not belong. The narrow table was also tried in search’s fuzzy-name stage. Generic names tie in their hundreds at one similarity, and it changed the top result of 16 of 5,002 queries for a 17% gain on one stage. Removed.

A recommendation that was wrong. The remaining cost is dominated by the typo fallback, which fires on 69% of keystrokes and is 52% of autocomplete CPU; much of what it appends looks like filler - for 389 congress st po, nine other towns’ Congress Streets after the one correct address. Firing it only when the prefix match found nothing would have made autocomplete **3.3 times** cheaper, and the 150 autocomplete accuracy cases did not move. The fuzz set found the hole. Walker Ci is one character off “Walker Corner”, but it still prefix-matches one row - Walker Heights Circle, 166 km away - so the fallback was suppressed, and the right town had been fourth, among the “filler”. The premise was that a typo makes the prefix match find nothing; sometimes it makes it find the wrong thing instead. One case in 3,360, inside a fuzz score that read 66.7% both times, found only by comparing every case individually. The change was reverted and the query is pinned by a test.

Key result

The largest gain on offer was declined. A 3.3-fold reduction was available for the loss of one case in 3,360, and the project’s position is that accuracy is what pgeo is for. The figure reported here, 36%, is what remained after every change that altered an answer was taken back out.

3.16.4 Capacity Same harness, corpus, session mix, ramp and latency targets as Section 3.4, pure-SQL front end, the same memory budgets.

Table 56. Concurrent users within the latency targets, before and after. Each configuration moved up one step of the ramp, the unconstrained one two; the ramp has no steps between, so each figure is a floor.

Configuration	Before	After	Change	Worst endpoint at the new limit
P1 - 1 vCPU, 1.0 GB	24 users	32	+33%	54% of its target
P2 - 2 vCPU, 1.5 GB	48 users	64	+33%	39%
P4 - 4 vCPU, 2.0 GB	96 users	128	+33%	45%
PM - unconstrained	128 users	192	+50%	21%

Because the “before” figures were two days old, the old functions were reinstalled and P2 was run again on the same day: the old engine held 48 users and failed at 64 with its worst endpoint at 1.44 times target, exactly as published, where the new engine passes 64 at 0.39. At that load the old engine’s autocomplete p95 was 360 ms against its 250 ms target; the new one’s is 98 ms. The gain is the code, not the day.

This changes the comparisons of Sections 3.14 and 3.15. On two cores pgeo now holds **64 users, level with Nominatim**; the cost of the all-SQL query path falls from four times to **three times against Pelias**, and from 2.7 to two times against Photon. Unconstrained it reaches Pelias’s 192, though that flatters pgeo: Pelias’s unconstrained run was held to one API worker (Section 3.4).

What is left is the typo fallback - two-thirds of the remaining autocomplete cost, and, as Section 3.16.3 shows, doing real work. Making its trigram recheck cheaper (10,221 index candidates for 1,543 survivors) is the open problem.

4. Discussion

Why pgeo is more accurate. The difference is in how text meets data. Elasticsearch matches analysed tokens; a token with a typo or an unexpected abbreviation contributes nothing, and the admin lookup requires exact names. pgeo compares normalized whole strings by trigram similarity, expands abbreviations before comparing (so “moountain rd” meets “mountain road”), and scores the queried town by distance as well as by name. It also knows the data is one state, which lets it use tie-breaks (distance to the focus point or the named town) that a planet-scale engine cannot assume.

Why Pelias is faster. Elasticsearch executes text matching as compiled index lookups and caches aggressively, and Pelias parallelizes across services. pgeo’s PL/pgSQL runs several candidate queries and scoring steps per request in an interpreted language inside one backend.

Is there still a reason to use Pelias? Yes, in four situations: (1) many concurrent users per server, where Pelias’s four-fold throughput per CPU saves hardware once its 7.17 GB memory floor is paid; (2) coverage beyond one or two states, for which Pelias is designed and pgeo is untested – and where the data-volume curve (Section 3.6.1) shows pgeo losing capacity about twice as fast as Pelias as names are added; (3) international addresses and multilingual names (libpostal, Pelias language support); (4) when a maintained upstream project matters more than accuracy, since pgeo is this project’s own code. For the deployment studied here (Maine, a handful of users, messy input, LANCER), none of these applies.

What the other two engines say about the SQL question. The point of building Photon and Nominatim was to find out whether Pelias flattered pgeo, and the answer is: on capacity, no, and in an unexpected direction. Photon is the engine built for the endpoint pgeo is worst at, and it does answer a keystroke two to three times quicker than pgeo – but it holds fewer concurrent users per core than Pelias, so it does not replace Pelias as the yardstick. Nominatim, the engine whose architecture is nearest to pgeo’s, comes closest of all: 64 users to pgeo’s 48. That comparison is the most informative one in the study, because Nominatim and pgeo differ in one thing only. Both compute and index inside PostgreSQL; Nominatim’s

search is a Python application issuing queries, pgeo's search is the query. As first measured, moving that last step into SQL cost a third of the capacity - and Section 3.16 shows that third was not inherent: it was a filter function that could not be inlined, result records built for rows about to be discarded, and a one-letter prefix expanded inside an index scan. With those removed pgeo holds the same 64 users Nominatim does. Set against the four times pgeo gives up to Pelias, this locates the expense precisely: most of it is not "SQL instead of compiled code", it is "a general-purpose relational engine instead of an inverted index". The rest turned out to be ordinary inefficiency in code nobody had profiled, which is a more useful finding than a fixed tax would have been: the SQL-only design carries no measurable penalty against its nearest architectural neighbour, and a threefold one against a purpose-built search engine.

Why the accuracy column flatters pgeo against the OpenStreetMap engines. pgeo answers 95.8% where Photon answers 73.7% and Nominatim 61.7%, and it would be wrong to read that as three engines ranked by quality. Photon and Nominatim index OpenStreetMap and nothing else - Photon's index is exported from a Nominatim database, and Nominatim imports OSM - so neither can hold the 780,260 OpenAddresses E911 points or the 76,582 Overture places that most of the test set is drawn from. Venue accuracy tells the story plainly: 94.2% for pgeo, 19.1% for Photon, 8.9% for Nominatim, because those venues are not in their indexes to be found. Nominatim's median positional error on the addresses it does return is zero metres. What pgeo can honestly claim against these two is not that it searches better, but that it can carry data they cannot. Two of the differences are real engine properties rather than data: Nominatim has no autocomplete endpoint, and neither engine expresses uncertainty, which for a dispatch application is disqualifying independent of speed.

The tuning caveat. pgeo was tuned in rounds that started from this test set's failing cases; Pelias was run as published. The fuzz rounds (corruptions of queries the tuning never saw as such) and the source-derived ground truth limit the effect, but part of the 20-point gap may be fit to the test set. The decisive check is real queries neither engine was tuned on: running both engines side by side on the query host and scoring the first weeks of actual searches (Next steps).

Caution

pgeo was tuned against the same test set that measures it here, and Pelias was not. The fuzz rounds and source-derived ground truth limit the effect, but treat the 20-point gap as an upper bound until both engines have been scored on real queries neither has seen.

Threats to validity.

- The accuracy set comes from the same sources both engines load; it measures finding what is in the data, not whether the data is correct. Eight miss cases are test-set errors.
- Load tests ran on one workstation with CPU pinning and memory limits standing in for VM sizes; absolute numbers on other hardware differ (Section 3.11 validates on the production VM).
- At 1-4 users the per-endpoint p95 rests on few requests; the ramp's decisions at higher user counts rest on thousands.
- One pgeo run (api-svc-P4) was interrupted by a host event: the Docker daemon was stopped at 04:32 on 2026-09-19, unrelated to the test, which ended the ramp at 32 users. That configuration was re-run from the start, and the results here are the complete re-run; no truncated run contributes a number to this report.
- The first Pelias data-volume run failed because Elasticsearch was not ready after the previous run's restart; it was repeated with a readiness wait.

5. Conclusions

Which tool suits which situation, and what the evidence supports saying. The recommendations below follow from the measurements rather than from a preference for either architecture.

Table 57. Which tool for which scenario.

Scenario	Recommended	Why
A few users on a small VM or VPS (this project's target)	pgeo, pure SQL	meets all targets from 1.4 GB and a quarter core (Section 3.12); most accurate; 2-3 containers
Messy input: typos, variants, venues, impossible places	pgeo	87% vs 30% on typos; misses handled 95% vs 41%
Structured US addresses for another system (LANCER)	pgeo	/v1/address; nearest street address for venues
Hundreds of concurrent users per vCPU	Pelias	three times the throughput per CPU (four before the optimization of Section 3.16)
Multi-state or national coverage	Pelias today	built for planet scale; measured: pgeo loses four times its capacity across Maine's data range where Pelias loses two (Section 3.6.1)
Fewest components, data in PostgreSQL (G6)	pgeo, pure SQL	database plus a stateless gateway
Existing Pelias clients	either	pgeo passes the compatibility contract
Type-ahead above all, OpenStreetMap data sufficient	Photon	53 ms per keystroke on two cores and 1.84 GB resident, the lightest measured; but no confidence score, and no OpenAddresses or Overture (Section 3.14)
A general OpenStreetMap geocoder, no type-ahead needed	Nominatim	exact when it answers (0 m median on addresses); but silent on 39% of queries and 8% correct at autocomplete, having no such endpoint (Section 3.15)

In one paragraph. For the service this project set out to build – Maine data, a handful of users, input typed by people who misspell things, and a preference for as few moving parts as possible – the PostgreSQL geocoder is the better instrument on every axis that matters here. It answers 95.8% of 1,560 test queries against 75.5% for Pelias, and the gap widens exactly where real input is worst: typos, half-remembered venue names and places that do not exist. It does that from two containers and 1.40 GB of memory, where Pelias needs six containers and 7.17 GB, which is the difference between the cheapest plan a provider sells and a mid-range server. And it speaks the Pelias API closely enough that an existing client cannot tell the difference (28 of 28 contract cases). What it gives up is throughput – a quarter of Pelias's users per CPU – and planet scale, neither of which this deployment needs. Measuring two further engines put that price in proportion: against Nominatim, whose architecture differs from pgeo's only in that its search runs in Python rather than in SQL, the cost is not four times but a third (64 users to 48) – and profiling the query path then closed that third entirely, with every one of 7,380 recorded queries returning byte-identical results (Section 3.16). What remains against Pelias, three times, is the inverted index, not the SQL. One caveat belongs in the last sentence rather than a footnote: pgeo was tuned against the set that scores it, so the 20-point accuracy gap should be read as an upper bound until both engines meet queries neither has seen. The capacity and memory findings carry no such caveat.

None of this rests on the test bench alone. Both engines were deployed to the machine that will actually serve the requests and measured again through the production HTTPS path: both meet the three-user target with the worst endpoint at under a seventh of its budget, Pelias holds 384 concurrent users there and pgeo 128, and pgeo holds 32 on a 1 GB machine costing about \$5 a month. The conclusions above are what the deployed service does, not what a benchmark suggested it might.

6. Future work

Each item below is given three things a reader needs before deciding to do it: roughly how much work it is, how likely it is to pay off, and what the payoff would be worth in practice. The estimates are the author's, and the ones marked speculative are the ones to distrust.

Work	Effort	Plausibility	What it is worth
------	--------	--------------	------------------

Table 58. Work not done, with the effort, the likelihood it pays off and the value of finishing it.

Work	Effort	Plausibility	What it is worth
Autocomplete speed: prepared statements, fewer candidate queries, a GiST trigram index for ordered similarity	2-3 days	High – autocomplete is measurably the binding endpoint on every configuration (Figure 14)	The clearest remaining win. A factor of two here roughly doubles users per vCPU and narrows the one axis where Pelias leads
Authorization (Phase 11): hashed API keys at the edge (done, Section 3.13.2), then single sign-on for the people behind the clients	weeks for SSO	Certain	Keys name machines; SSO would name people. The blocking half is closed: a device on the LAN can no longer resolve a client address without a key
Published container images (Forgejo and GitHub registries): a <i>pre-processor</i> image that builds the database from raw sources on a workstation, an <i>all-in-one</i> image with the pre-processors included for slower but centralised deployment, and a pipeline that moves a built database from the workstation image to a VPS running the server side	3-5 days	High – every piece exists (loader, dump, Ansible restore, pinned images); this is packaging and a transport	Turns “clone the repo and run four scripts” into <code>docker pull</code> , which is what adoption by other counties and states depends on. Design open: see docs/DOCKER_IMAGES.md
Atomic online restore: restore into a second schema and rename, so a data update does not take the service down	1 day	High – the build already does exactly this locally	Turns a weekly refresh from an outage into a non-event. The single most useful automation left (Section 7.3)
Hysteresis test: ramp past the breaking point, then back down, and measure recovery	Half a day	Certain to produce an answer	Closes the one operational question Section 7.4 cannot answer: whether the service comes back on its own
Shadow comparison on real traffic: both engines on the query host for weeks, scored on real searches	2 days to set up, then waiting	High	The only way to settle the tuning caveat in Section 4. Until then the 20-point accuracy gap is an upper bound
pgeo data-volume curve: subset builds (D1-D5) as done for Pelias	1 day	Certain	Answers whether pgeo’s advantage survives more data – the obvious challenge to a one-state result
New Hampshire: build both engines for two states	2-3 days	Medium – the pipeline is state-agnostic, but ranking is tuned on Maine	The real test of whether “one state, tested” generalises
Horizontal scale-out: a read replica behind a load balancer, same ramp	1 day	High for throughput, none for latency	Shows whether pgeo answers the “but it does not scale” objection with \$5 machines (Section 7.5)

Work	Effort	Plausibility	What it is worth
Compare against Photon (done, Section 3.14)	-	-	Measured: Photon holds 2.7x as many users as pgeo on the same two cores and answers a keystroke in 53 ms against pgeo's 103-189 ms, but holds fewer users than Pelias, so the 4x figure stands. Its index is OpenStreetMap-only and it returns no confidence score
metaphone fallback for phonetic misspellings	Half a day	Medium – it should help at high corruption, and it might help nothing	A few points at fuzz levels F4-F5 (Section 7.1), or a clean negative result
Category taxonomy: map source categories onto Pelias's	1-2 days	High	Closes the last documented API difference, so <code>categories=food</code> behaves as a Pelias client expects
libpostal model update (Senzing libpostal-data)	1 day	Low for pgeo, medium for Pelias	pgeo's rule parser already beats current libpostal (95.8% against 94.0%), so this mostly matters to the Pelias arm
Test set: remove the eight false misses, add real queries	Half a day	Certain	The eight are known errors inflating pgeo's miss score; removing them makes the headline number defensible
Continuous checks: accuracy gate and compatibility contract on every build	Half a day	Certain	Both exist and pass; this is only wiring them into a hook

6.1 Would OpenResty or Omnigres be worth trying?

Both were considered and rejected when the HTTP layer was chosen (`docs/HTTP_OPTIONS.md`), and the question is whether the results since then change that. They would take the “no application code” idea further than PostgREST does, in opposite directions.

Omnigres (`omni_httpd`) puts the HTTP server *inside* PostgreSQL: its workers are database backends and its route handlers are SQL. It is the literal form of this project's guiding principle – there would be no second container at all, and the memory floor would drop by PostgREST's 0.15 GB. Against that: it is a third-party C extension running in the database process, which is precisely the risk the project avoided by insisting on core and contrib extensions only, and its authors describe HTTPS and HTTP/2 as still maturing. **Worth testing? Not for this deployment.** The gain is 0.15 GB on a 1.4 GB floor and one fewer container, against putting unvetted C in the same process as the data. It would become interesting if the project ever needed to drop below a 1 GB machine, which nothing here suggests it does.

OpenResty with pgmoon keeps the gateway outside the database but replaces PostgREST with a few dozen lines of Lua in nginx – and nginx is already on the host as the edge. That is genuinely attractive on paper: one fewer process, no PostgREST parameter-naming constraints, and the `boundary.circle` rewriting that currently needs chained nginx `map` directives (Section 3.8) would become three lines of Lua. Against it: a non-standard nginx build, and Lua that this project would own and have to test – the opposite of the argument that PostgREST needs no code. **Worth testing? As an experiment, yes – and only if PostgREST's URL grammar becomes a real obstacle.** It is perhaps two days to a working prototype and a like-for-like ramp. The measurement that would justify it is latency: PostgREST added no measurable overhead at or below 4 vCPU (Section 3.4), so the honest expectation is that OpenResty would win on elegance and nothing else.

The broader point is that the HTTP layer is not where this system spends its time. Section 3.4 shows the

database is the bottleneck on every configuration, so a faster or lighter gateway moves a number that is not currently limiting anything. Both options are worth knowing about; neither is worth doing before autocomplete speed and authorization.

7. Questions and answers

Questions raised while reviewing this work, answered with measurements where a measurement was possible and with a clear statement of what was not tested where it was not.

7.1 Would a soundex-style filter help when users cannot spell a name?

Short answer: not soundex, and not as a replacement for what is already there – but *metaphone* as a *fallback* is worth an experiment.

pgeo already handles phonetic misspellings through trigram similarity. Five invented misspellings of Maine towns all resolve today:

Table 59. Phonetic misspellings against the deployed pgeo. All five resolve, but two are weak: “Kenebunc” is found with low confidence, and “Skowhegin” ranks a venue above the town.

Typed	pgeo returns	Confidence	Trigram similarity to the true name
Kenebunc	Kennebunk, ME	0.32	0.46
Skowhegin	The Cabin- Skowhegan, Skowhegan, ME	0.76	0.54
Mooshead	Moosehead, Northeast Somerset, ME	0.48	0.58
Bidefford	Biddeford, ME	0.59	0.54
Naragansett	Naragansett Avenue, Buxton, ME	0.96	0.79

So the gap is not *finding* these names but ranking and scoring them confidently. That is where a phonetic key could help, and it is also where soundex fails badly. Soundex keeps roughly four characters of an English-phonetic code, which is far too coarse for place names:

Table 60. How coarse each phonetic encoding is on this data. Soundex puts 18 distinct towns – North Bancroft, Northport, North Haven, Northfield and fourteen more – into one bucket.

Encoding	Distinct codes for 1,466 Maine town names	Names sharing a code with another
soundex	885	293 buckets
dmetaphone	987	735 names
metaphone(name, 16)	1,428	few

Pros. A phonetic key catches the case trigram similarity misses: a spelling that *sounds* right but shares few three-character slices with the target. It costs one functional index and no new component (fuzzystmatch is already installed).

Cons. Soundex would actively harm this data: collapsing eighteen “North ...” towns into one bucket invites exactly the ambiguity the confidence work in Section 3.2 was built to remove. It is also English-only, and it encodes the whole multi-word string rather than the distinctive part.

What would be worth testing: `metaphone(name, 16)` as a *fallback* – consulted only when trigram finds nothing above threshold, never as a parallel ranking signal – with the fuzz set (Section 2.4) as the measure. Plausible gain: a few points at the highest corruption levels (F4, F5), where pgeo currently scores 45-49%. Effort: half a day. It should be measured, not assumed: the same gate that protects every other tuning step would decide it.

7.2 Are pgeo's SQL routines functions or stored procedures? Why, and what does it cost?

They are functions: 41 of them, and no procedures at all.

PostgreSQL has had true stored procedures since version 11 (`CREATE PROCEDURE`, invoked with `CALL`), and they differ from functions in one way that decides the matter here: a procedure can manage transactions but cannot return a result set to a caller in a `SELECT`. Everything pgeo does is a read that returns rows or JSON, so functions are the only fit. Two consequences follow.

Performance. Functions can be declared `STABLE` and `PARALLEL SAFE`, which pgeo's are. That lets the planner call them inside a query, reuse a cached plan for the session, and consider parallel workers. A procedure would force a round trip per call and give up plan reuse. PL/pgSQL caches a prepared plan for each statement inside a function after the first few executions, which is why the first request to a fresh connection is slower than the rest – visible in the warm-up excluded from every measurement in Section 3.

Flexibility. The cost is real and this project hit it repeatedly: `CREATE OR REPLACE FUNCTION` cannot change a parameter's name or type, and PostgREST addresses functions *by parameter name*. Renaming one argument means dropping and recreating it, and a stale overload silently shadows the new definition. pgeo's build therefore drops and recreates the whole `geocode_api` schema on every apply (`pgeo/sql/050_api.sql`), which is simple and total. The lesson is in Table 25: several tuning rounds were slowed by overloads left behind after a signature change.

7.3 How are data updates prepared on a fast machine and installed into production?

This is how the project already works, and it is the reason a 1 GB server is viable at all (Section 3.5): **the query host never sees the raw data or the build tools.**

1. Build on a workstation: `pgeo-load build` loads every source, assigns the admin hierarchy, deduplicates, indexes and swaps the new schema in atomically. 13 minutes, 5.5 GB peak.
2. `scripts/pgeo_dump.sh` produces a compressed dump of just the three schemas (117 MB) and refuses to run if the database reports an unstamped engine version.
3. Ansible ships the dump, restores it, recreates extensions and the read-only role, runs `VACUUM (ANALYZE)` and a smoke test (`infra/ansible/roles/pgeo_runtime`).

Pros. The production machine needs no build capacity, no source data and no network access to the data providers; a bad build never reaches it; and rollback is restoring the previous dump. **Cons.** The restore is not online – the database is unavailable while it runs – and the dump is a full copy each time rather than a delta, so a weekly refresh moves 117 MB whether ten addresses changed or ten thousand.

Can it be semi-automated? It already is: `scripts/rebuild_all.sh` `deploy` runs the whole chain. Making it *fully* automatic needs two things it does not have: a restore into a second schema followed by an atomic rename, so the service stays up (the build already does exactly this locally, so the pattern is proven), and a scheduled trigger with the accuracy gate as the gatekeeper – no dump ships unless the gate passes. That is perhaps a day's work and it is the single most useful piece of automation left.

7.4 What happens when load exceeds the VM's resources?

The two engines fail in different ways, and this was measured rather than reasoned about (Section 3.12).

pgeo degrades. Every failed trial in the minimum-server search failed on latency, never on memory: the database kept answering, just too slowly. Requests queue at the connection pool, p95 rises, and nothing dies. That is the gentler failure and it is a property of the architecture – one process per connection, a small working set, no in-memory model to lose.

Pelias stops. Below a threshold its services do not degrade, they exit: `placeholder` is OOM-killed at 0.4-0.55 GB, `pip` at 0.55 GB, interpolation crash-loops at 1.8 GB, Elasticsearch is killed at 0.8 GB, and `libpostal` will not start below 2 GB at all.

Can it be detected and the user warned? Yes, and cheaply, because the two signatures are different. Rising p95 with no errors is pgeo under pressure: alert on the p95 of autocomplete, which is the first end-point to cross its target every time (Section 3.4). Container restarts or OOM kills are Pelias under pressure: alert on restart counts. The load harness already computes both, so the thresholds are known rather than guessed.

Do they recover when pressure backs off? For pgeo the mechanism says yes – the queue drains and no state is lost – but **this was not measured**: every ramp in this study increases load and stops at the breaking point, so nothing here tests the way back down. For Pelias, a service that was OOM-killed restarts with cold caches, so recovery is slower than the failure. A hysteresis test (ramp up past the limit, then back down, and measure how long p95 takes to return) is a half-day of work and is listed in Section 6.

7.5 Can pgeo scale horizontally the way Pelias does?

Yes in principle, and more cheaply than Pelias, but it has not been tested.

The workload is entirely read-only, which is the case PostgreSQL scales out best. Streaming replication to read replicas with PostgREST in front of each – or one PostgREST per replica behind a load balancer – needs no application changes, because nothing in the query path writes. Each replica is a full copy of a 654 MB database, which is small enough that a replica costs a 1 GB machine (Section 3.12): horizontal scaling here is adding \$5 VPSs.

Pelias scales the same axis with Elasticsearch replicas and more API instances, which is a well-trodden path with real operational tooling behind it. Two honest differences: Elasticsearch shards a large index across nodes, which pgeo has no equivalent of and does not need at 654 MB; and Pelias's path is documented and widely run, where pgeo's is inferred from how PostgreSQL behaves rather than demonstrated.

The limit worth noting is that this buys throughput, not latency. Section 3.4 shows pgeo is CPU-bound per request, so replicas multiply the users served without making any single query faster. The work to prove it – one replica, a load balancer, and the same ramp – is about a day, and it is the natural follow-on to the capacity results.

Acknowledgements

This study measures pgeo against **Pelias**, and would not exist without it. Pelias was built at Mapzen from 2013 and has been maintained since January 2018 by Geocode Earth, under the MIT licence, with a community of contributors. It supplied the API pgeo implements, the tooling that prepares two of the data sources, and the reference answers that decided what “correct” means in Section 3.1. Where this report shows pgeo ahead, it is ahead on a problem Pelias defined (Section 1.5).

The open data this work depends on is maintained by the OpenStreetMap contributors, the OpenAddresses project, Who's On First, the USGS, the US Census Bureau and the Overture Maps Foundation. Each carries its own licence; see NOTICE in the repository.

References

Numbered in order of first citation. Everything external to this project that the report asserts is listed here; measurements are the project's own and are reproducible from the repository (Appendix A).

1. Pelias: an open-source geocoder. <https://pelias.io> and <https://github.com/pelias/pelias>
2. Elasticsearch. Elastic. <https://www.elastic.co/elasticsearch>
3. Photon: an open-source geocoder for OpenStreetMap data. <https://github.com/komoot/photon>
4. Nominatim architecture overview, release 5.3 documentation. <https://nominatim.org/release-docs/latest/develop/overview/>
5. Pelias open-source project update, 2 January 2018 (the Mapzen shutdown announcement). <https://pelias.io/announcements/2018-01-02-pelias-update/>
6. Geocode Earth: the company founded in January 2018 by former Mapzen staff, which maintains Pelias. <https://geocode.earth/opensource/>
7. A trip down memory lane, Geocode Earth, 2020. <https://geocode.earth/blog/2020/memory-lane/>
8. Pelias is released under the MIT License. <https://github.com/pelias/pelias/blob/master/LICENSE>
9. PostGIS Tiger Geocoder, chapter 12 (Extras). <https://postgis.net/docs/Extras.html>
10. pgGeocoder: a Japanese geocoder implemented entirely in PostgreSQL functions. <https://github.com/mbasa/pgGeocoder>
11. PostGIS Tiger Geocoder 2025.1 release notes, June 2026. <https://postgis.net/2026/06/PostGIS-Tiger-Geocoder-2025.1/>
12. address_standardizer: a PostGIS extension that parses addresses using lexicon, grammar and rule tables. https://postgis.net/docs/Address_Standardizer.html
13. osmgeocoder: an OpenStreetMap and OpenAddresses geocoder using PostGIS, pg_trgm and fuzzys-trmatch. <https://github.com/dunkelstern/osmgeocoder>
14. fuzzys-trmatch: soundex, metaphone, double metaphone and Levenshtein. PostgreSQL documentation. <https://www.postgresql.org/docs/18/fuzzys-trmatch.html>
15. moofish32/postgis-geocoder: a Docker packaging of the PostGIS TIGER geocoder. <https://github.com/moofish32/postgis-geocoder>
16. MattTriano/postgis_geocoder: a Docker and Python packaging of the PostGIS TIGER geocoder. https://github.com/MattTriano/postgis_geocoder
17. The road to Nominatim 6, July 2025. <https://nominatim.org/2025/07/14/roadmap-nominatim6.html>
18. OpenCage geocoding foreign data wrapper for PostgreSQL (calls a remote API from SQL). <https://blog.opencagedata.com/post/geocoding-in-postgresql>
19. libpostal: a C library for parsing and normalising street addresses, using statistical NLP. <https://github.com/openvenues/libpostal>
20. Grafana k6: a load-testing tool with scripted virtual users. <https://k6.io>
21. pg_trgm: trigram-based text similarity and indexing. PostgreSQL documentation. <https://www.postgresql.org/docs/18/pgtrgm.html>
22. Who's On First: an open gazetteer of places. <https://whosonfirst.org>
23. OpenAddresses: a global collection of open address data. <https://openaddresses.io>
24. USGS Geographic Names Information System (GNIS), Domestic Names. <https://www.usgs.gov/usb-board-on-geographic-names>
25. US Census Bureau, ZIP Code Tabulation Areas (ZCTA) gazetteer. <https://www.census.gov/geographies/reference-files/time-series/geo/gazetteer-files.html>
26. DuckDB, used in the pgeo build to read Parquet and CSV sources. <https://duckdb.org>
27. GDAL/OGR (ogr2ogr), used in the pgeo build to load spatial formats. <https://gdal.org>
28. United States Postal Service, Publication 28: Postal Addressing Standards. <https://pe.usps.com/text/pub28/welcome.htm>
29. United States Postal Service, ZIP+4 product and licence terms. <https://postalpro.usps.com/address-quality/zip4>

Appendix A. Reproducing this study

Everything in this report can be rebuilt from the repository. The authoritative runbook is `docs/REBUILD.md`: requirements, every stage with its commands, expected results and times, the load tests and experiments, and troubleshooting. `scripts/rebuild_all.sh` runs the stages end to end (data, Pelias, pgeo, verification, report; deployment on request):

```
scripts/rebuild_all.sh          # check, data, pelias, pgeo, verify, report
scripts/rebuild_all.sh deploy  # ship both engines to the query host
```

The report itself:

```
scripts/build_report.sh          # rebuild from saved results
scripts/build_report.sh --refresh # refresh live snapshot and contract test first
scripts/build_report.sh --refresh-accuracy # also re-run the accuracy and fuzz sets
```

Inputs: `report/inputs.toml`. The build renders this document twice from one template, as Markdown (`docs/REPORT.md`) and, through pandoc and XeLaTeX, as PDF (`docs/REPORT.pdf`); figures are published as PNG at 300 dpi (used here) and SVG (editable) in `docs/report_figures/`. Load tests: `tests/load/run_matrix.py` (Pelias) and `tests/load/run_matrix_pgeo.py` (pgeo); the minimum-server search: `tests/load/find_floor.py`; accuracy: `tests/accuracy/`; compatibility: `tests/compat/`.

The repository versions three components separately – the project (`VERSION`), pgeo (`pgeo/pyproject.toml`) and the data preparation tools (`prep/pyproject.toml`) – under semantic versioning, each with its own changelog and git tags (`vX.Y.Z`, `pgeo-vX.Y.Z`, `prep-vX.Y.Z`); the tags for work that predates this scheme were applied retroactively. A full rebuild run end to end against the accuracy gate scored 95.7% and passed, which is the check that a rebuilt system reproduces the results in this report.

Appendix B. Questions asked during the project

Every question the project owner asked during the work, with a short answer and where the report answers it in full.

Table 61. Questions asked during the project, with short answers and the sections that answer them in full.

Question	Short answer	Section
Can the search capabilities be reproduced with a web API straight from PostgreSQL/PostGIS and extensions?	Yes: pgeo, SQL functions behind PostgREST, more accurate than Pelias on Maine data	2.1, 3
Is Overture address data worth using, compared with OpenAddresses?	Not for Maine: the same E911 points packaged twice; places were loaded	2.2
Did the VM go on the Proxmox host or locally?	The service runs on VM 120 on the Proxmox host; builds and controlled tests run on the workstation	1.2
Is an OpenAddresses API key needed?	No: the statewide Maine file downloads without one	1.2
Can caching affect later results?	Caches are warmed the same way for both engines and warm-up is excluded; results are warm steady state	2.5
Is all the testing only with Pelias?	No: every test ran on both engines (pgeo with both front ends)	2.5

Question	Short answer	Section
How long do the tests take?	Accuracy 3-5 min per engine; a capacity matrix 2-7 hours; the minimum-server search 2-3 hours	Appendix A, docs/REBUILD.md section 4
What are the latency targets?	p95: autocomplete 250 ms, search and structured 750 ms, reverse 400 ms; errors below 1%	2.5
How many users does the ramp go to?	1 up to 512, stopping when a configuration breaks	2.5
Does pgeo use PostgreSQL's parallelism across cores?	Yes between queries (one backend per connection; capacity scales with cores); intra-query parallelism is off for short queries and not measured	2.7, 3.4
Can pgeo replace Pelias for existing clients, with extensions?	Yes for the documented API (27 of 27 contract cases), plus /v1/address	3.8, 3.10
Is testing on the Proxmox VM needed for accurate results?	Both: the workstation isolates the engines for a fair comparison; the VM measures the real service	2.5, 3.11
Which machine does the builds?	The workstation (Ryzen 5 3600, 31 GB); builds are shipped to the VM as a snapshot and a dump	1.2, 2.3
Do the platforms now have feature and accuracy parity?	Feature parity for the documented API; pgeo is more accurate; Pelias has about four times the throughput per CPU	3.10
Is there still a reason to use Pelias?	For many users per server, multi-state or international coverage, or a maintained upstream	4
Can USPS ZIP+4 data improve the addresses?	Yes technically, but it is licensed for internal use on one computer; declined	3.9
Does the newer libpostal model (Senzing libpostal-data) help in the US?	Not evaluated in this study; pgeo's rule parser already beats the current libpostal (95.8% vs 94.0%), so it would matter mainly for Pelias	6
Can a non-address search result return the nearest street address?	Yes: /v1/address returns the nearest address point and its distance	3.9
Which engine performs better, and is pgeo good enough on the same or fewer resources?	Pelias is faster; pgeo meets the 3-user goal in a quarter of the memory and is more accurate	Summary, 3.4, 3.10
How do more data layers affect performance?	Pelias: addresses barely; OSM and Overture places cost one ramp step each	3.6
What are the arguments for and against the external HTTP options?	PostgREST chosen; Omnigres rejected (code in the database); OpenResty as fallback	2.1
What does each platform need to build and to run, and what loads and data can it support?	Sizing guide by load; operating and build requirements	3.5
What features does each platform provide?	Feature matrix and parity table	3.9, 3.10
How small can a server be for 3 concurrent users, and how many users does it then scale to?	See Section 3.12	3.12

Question	Short answer	Section
How do the two platforms compare on security, and how should each be deployed?	pgeo authenticates its data store and its API role cannot write; Pelias's Elasticsearch has no credentials, so its loopback binding is the whole control. Both sit behind the same edge, which now authorizes callers by hashed API key	3.13, docs/SECURITY_DEPLOYMENT.md
What do the measured floors mean in shared-CPU VPS plans (Linode, DigitalOcean, Vultr, Hetzner and others)?	pgeo runs on a 1 GB plan (confirmed on a real VM, 32 users); Pelias needs an 8 GB plan, about ten times the price	3.5 (Table 24), 3.12

Appendix C. pgeo storage

The schema holds 847 MB in tables and their indexes. Dropping the tables the build needs only while loading is what keeps it that small: on the build where it was measured it took the database from 971 MB to 674 MB. The database file is larger – 868 MB when this snapshot was taken – because the atomic schema swap leaves the previous schema in place until it is dropped and vacuumed, and that is the number a disk has to hold. Trigram indexes on names dominate: they are what makes a misspelled query answerable, and they cost roughly as much space as the rows they index.

Table 62. pgeo tables (including their indexes).

Table	Size (incl. indexes)
feature	805 MB
admin	22 MB
street_name	16 MB
ac_prefix	3 MB
town	0 MB
build_info	0 MB

Table 63. Largest pgeo indexes.

Index	Size
feature_gid_idx	52 MB
feature_geom_idx	36 MB
feature_addr_idx	33 MB
feature_addr_street_hn_idx	23 MB
feature_addr_hn_idx	23 MB
feature_pkey	19 MB
feature_tokens_idx	18 MB
feature_name_trgm_idx	13 MB
feature_layer_idx	6 MB
feature_name_idx	6 MB
street_name_pkey	5 MB
street_name_trgm_idx	4 MB